



UNIVERSITY OF PADOVA

DEPARTMENT OF INFORMATION ENGINEERING “DEI”

MASTER THESIS IN COMPUTER ENGINEERING

WHEN TIC-TAC-TOE WASN'T HARD ENOUGH: A REINFORCEMENT LEARNING JOURNEY IN ULTIMATE TIC TAC TOE

SUPERVISOR

PROF. GIAN ANTONIO SUSTO
UNIVERSITY OF PADOVA

MASTER CANDIDATE

ENRICO D'ALBERTON

STUDENT ID

2093708

ACADEMIC YEAR

2024-2025

Acknowledgments

I would like to express my sincere gratitude to Professor Gian Antonio Susto, whose work and teaching inspired my interest in this fascinating subject.

I am also deeply thankful to Alberto Sinigaglia and Alessio Arcudi, the two PhD students who guided me through this long and challenging project with patience and dedication. Without their support, none of this would have been possible.

I thank my family: my father Lorenzo, my mother Loredana and my brother Giulio, for their support at every step throughout this journey.

A special thanks goes to Sara, for always being by my side.

I'm grateful to Pietro for being a constant guide and source of support.

Finally, I want to thank all the friends I met during these years: they are probably the most valuable thing university has given me.

Abstract

This thesis presents an investigation into the application of Reinforcement Learning (RL) techniques to the game of Ultimate Tic Tac Toe (UTTT), a complex extension of the classic Tic Tac Toe. The game’s unique combination of local and global strategic dynamics makes it an ideal testbed for evaluating the capabilities of modern RL algorithms in navigating large, structured action and state spaces. After providing a theoretical foundation in reinforcement learning, game theory, and algorithmic strategy, the study focuses on the design, training, and evaluation of several RL agents, including Deep Q-Networks (DQN), Double DQN (DDQN), Advantage Actor-Critic (A2C), and Proximal Policy Optimization (PPO).

To address the complexity and strategic richness of UTTT, the thesis introduces a dedicated environment and training pipeline for self-play reinforcement learning. Each algorithm has been trained in isolation and evaluated against random agents to establish a performance baseline. Subsequently, the most promising models have been further refined using advanced techniques such as residual neural networks, data augmentation and hyperparameter tuning. Performance is measured using a combination of win/draw/loss statistics, Elo rating systems and round-robin tournaments between agents.

A key contribution of this work is the integration of best response training to evaluate the exploitability of learned policies. By training a separate best response agent using a DQN-based framework, the study reveals significant vulnerabilities in self-play agents and highlights the need for population-based training approaches to approximate Nash equilibria. The final DDQN-based model, enhanced with residual architectures and rotational data augmentation, achieves a win-rate of 85% against all the others agents trained, corresponding to a 72% relative improvement compared to the original developed model.

This thesis not only demonstrates the effectiveness of RL in mastering a non-trivial board game from scratch but also highlights the limitations of standard self-play in producing robust strategies. By combining insights from reinforcement learning, neural network design and game theory, this research contributes a methodical and reproducible framework for developing and evaluating competitive agents in discrete, turn-based games with high strategic complexity.

Contents

ABSTRACT	v
LIST OF FIGURES	ix
LIST OF TABLES	xiii
LISTING OF ACRONYMS	xv
1 INTRODUCTION	1
1.1 Motivation and Objectives	1
1.2 Scope and Approach	2
2 REINFORCEMENT LEARNING	3
2.1 History of Reinforcement Learning	4
2.1.1 Foundations of Learning: Animal Behaviorism and its impact	4
2.1.2 From Temporal Difference Learning to Contemporary Reinforcement Learning	6
2.1.3 The Deep RL Revolution	7
2.2 The Reinforcement Learning process	9
2.2.1 Markov Decision Processes (MDPs)	9
2.2.2 Value Functions and Bellman Equations	11
2.2.3 Optimal Policies and the Path to Maximization	13
2.2.4 Exploration vs. Exploitation Dilemma	15
2.2.5 Key Methods: Q-learning, SARSA, and Temporal Difference Learning	16
2.2.6 Value function approximation	21
3 ULTIMATE TIC TAC TOE	27
3.0.1 Game Rules	28
3.0.2 Related Works	29
4 THE ALGORITHMS BEHIND ULTIMATE TIC TAC TOE	31
4.1 Deep Q-Network	31
4.2 Double Q-Learning	32
4.3 Actor-Critic	34
4.4 Proximal Policy Optimization	35
4.5 Monte Carlo Tree Search	37

4.6	AlphaZero	39
5	GAME THEORY	43
5.1	Introduction to Game Theory	43
5.2	Basic Concepts of Game Theory	44
5.3	Type of games	46
5.4	Nash Equilibrium	47
6	EXPERIMENTAL SETUP	51
6.1	The Ultimate Tic Tac Toe environment	52
6.2	Environment Setup	54
7	RESULTS	57
7.1	Models setup	58
7.1.1	DQN Results	58
7.1.2	DDQN Results	60
7.1.3	A2C Results	61
7.1.4	PPO Results	62
7.2	Best Model Research	64
7.2.1	Free-for-All Tournament	65
7.2.2	Elo Rating	66
7.2.3	Additional improvements	68
7.3	Best response	76
7.4	Discussion of Results	78
7.4.1	Analysis	78
7.4.2	Future works	79
8	CONCLUSIONS	81
	REFERENCES	83

Listing of figures

2.1	Typical interaction between state, action, reward, agent and environment . . .	4
2.2	Policy Iteration schema	14
2.3	A Visual Comparison of SARSA and Q-Learning Algorithms	19
2.4	A Visual Representation of Temporal Difference (TD) Learning Methods . .	22
3.1	Board of Ultimate Tic Tac Toe	28
3.2	Visualization of UTTT's game dynamics	29
4.1	Visualization of Double DQN's update rule functioning	33
4.2	Visualization of the Actor-Critic structure	35
4.3	Visualization of MCTS steps	38
6.1	An example of a visual representation of the game state. The top section shows the board as seen by the player, while the bottom section illustrates the input format provided to the algorithms.	52
6.2	A visual representation of the game actions encoding.	53
7.1	Plot of the exploration rate.	59
7.2	This chart represent the performances of the DQN model assessed every 1000 iterations against a random agent on 1000 simulated games. In green, we represent the win rate of the trained agent. In blue, we represent the draw rate. In red we represent the loss rate.	59
7.3	This chart represent the performances of the DDQN model assessed every 1000 iterations against a random agent on 1000 simulated games. In green, we represent the win rate of the trained agent. In blue, we represent the draw rate. In red we represent the loss rate.	60
7.4	This chart represents the performances of the A2C model assessed every 1000 iterations against a random agent on 1000 simulated games. In green, we represent the win rate of the trained agent. In blue, we represent the draw rate. In red, we represent the loss rate.	62
7.5	Performance of the PPO agent assessed every 1000 iterations against a random agent on 1000 simulated games. The green bars represent the win rate, blue bars represent the draw rate, and red bars represent the loss rate.	63

7.6	Empirical outcome frequencies for each agent pairing, where rows represent the evaluated (“row”) agent and columns represent the opponent. Each cell visualizes the proportion of results—green for wins, gray for draws, and red for losses-computed 2000 games per matchup.	65
7.7	Expected-score advantage matrix: each cell shows (win%-loss%) for the row agent versus the column agent, color-coded from red (strong advantage) through white (neutral) to blue (strong disadvantage).	66
7.8	Bar chart of Elo scores by agent, sorted from highest (PPO_0E) to lowest (DQN).	67
7.9	Visual representation of the residual network exploited in our experiment. . .	68
7.10	This chart represents the performances of the DDQN_resnet model assessed every 1000 iterations against a random agent on 1000 simulated games. In green, we represent the win rate of the trained agent. In blue, we represent the draw rate. In red, we represent the loss rate.	69
7.11	This chart represents the performances of the PPO_0E_resnet model assessed every 1000 iterations against a random agent on 1000 simulated games. In green, we represent the win rate of the trained agent. In blue, we represent the draw rate. In red, we represent the loss rate.	69
7.12	Visual representation of rotational data augmentation. Starting from the original board configuration (0°), we generate three additional training samples by rotating the board by 90°, 180°, and 270°.	70
7.13	This chart represents the performances of the DDQN_resnet_data_aug model assessed every 1000 iterations against a random agent on 1000 simulated games. In green, we represent the win rate of the trained agent. In blue, we represent the draw rate. In red, we represent the loss rate.	71
7.14	This chart represents the performances of the PPO_0E_resnet_data_aug model assessed every 1000 iterations against a random agent on 1000 simulated games. In green, we represent the win rate of the trained agent. In blue, we represent the draw rate. In red, we represent the loss rate.	71
7.15	Empirical outcome frequencies for each agent pairing, where rows represent the evaluated (“row”) agent and columns represent the opponent. Each cell visualizes the proportion of results—green for wins, gray for draws, and red for losses-computed 2000 games per matchup. In this chart we added DDQN_resnet, DDQN_resnet_data_aug, PPO_0E_resnet, PPO_0E_resnet_data_aug. . .	72
7.16	Final Elo ratings for all agents.	73
7.17	Bar chart of Elo scores by agent, sorted from highest to lowest. In red there are the new trained models: DDQN_resnet, DDQN_resnet_data_aug, PPO_0E_resnet and PPO_0E_resnet_data_aug.	73
7.18	Performance of the DDQN_resnet_data_aug_2m1n model evaluated every 1000 iterations against a random agent over 1000 games. The green bars represent the win rate, blue bars the draw rate, and red bars the loss rate.	74

7.19 Empirical outcome frequencies for each agent pairing, where rows represent the evaluated agent and columns represent the opponent. Each cell visualizes the proportion of results—green for wins, gray for draws, and red for losses—computed over 2000 games per matchup. The newly added agent is DDQN_resnet_data_aug_2mln. 75

7.20 Final Elo ratings by agent, sorted from highest to lowest. In red is the top-performing model: DDQN_resnet_data_aug_2mln. 76

7.21 This chart represents the win and draw rate summed of the agents showed in the legend, different colors for different agents, against the best response under training every 1,000 iterations. 77

Listing of tables

5.1	Payoff matrix for the Prisoner's Dilemma	49
7.1	Final Elo ratings after iterative batch updates (K=20, 50 passes).	67
7.2	Performance of the final DDQN model against an MCTS agent with 1,000 simulations.	78

Listing of acronyms

RL	Reinforcement Learning
AI	Artificial Intelligence
MDP	Markov Decision Process
TD	Temporal Difference
PPO	Proximal Policy Optimization
A2C	Advantage Actor Critic
DQN	Deep Q-Network
DDQN	Double Deep Q-Network
UTTT	Ultimate Tic Tac Toe
MCTS	Monte Carlo Tree Search
ResNet	Residual Network
GPU	Graphics Processing Unit

1

Introduction

1.1 MOTIVATION AND OBJECTIVES

Reinforcement Learning (RL) is a core area of modern artificial intelligence, known for enabling agents to learn optimal behavior through direct interaction with an environment. Its strength lies in solving sequential decision-making problems without the need for explicit supervision, making it suitable for complex, dynamic tasks.

The objective of this thesis is to provide a comprehensive overview of the most relevant RL algorithms, with a focus on deep reinforcement learning methods that utilize neural networks for value approximation and policy learning. The second goal is to apply these algorithms to the game of Ultimate Tic Tac Toe (UTTT), a challenging environment characterized by spatial constraints, long-term dependencies and delayed rewards.

The study also incorporates a game-theoretic perspective, analyzing the strategic properties of the trained agents, their robustness against exploitative opponents, and their capacity to approximate equilibrium behavior. The focus is not limited to performance metrics but extends to the theoretical strength and resilience of the learned strategies.

1.2 SCOPE AND APPROACH

The thesis is structured to first build the theoretical and algorithmic foundation, followed by the practical application and evaluation of RL agents in the UTTT environment.

It begins with a formal introduction to the fundamentals of reinforcement learning, including Markov Decision Processes (MDPs), policies, value functions and the Bellman equations. The main algorithmic families are then presented, covering both value-based methods (such as Q-learning, DQN, and DDQN) and policy-based methods (such as A2C and PPO). Particular attention is placed on architectural choices, training techniques and regularization strategies, including experience replay, target networks, entropy bonuses, and data augmentation.

After establishing the RL background, the thesis introduces Ultimate Tic Tac Toe as the target environment. The rules of the game are formally described, and its suitability as a benchmark for RL is justified. The unique structure of UTTT introduces non-trivial constraints that make it significantly more complex than standard Tic Tac Toe, requiring agents to learn both local tactics and global strategy.

The implementation details of the agents, the self-play training setup, and the evaluation pipeline are then presented. Performance is measured using the Elo rating system, allowing for consistent comparisons between agents with different architectures and training schemes.

Eventually, the final chapters apply tools from game theory to analyze exploitability and evaluate how well agents resist best-response strategies.

The work aims to assess the effectiveness of deep RL algorithms in structured, adversarial environments such as UTTT. It highlights the impact of architectural choices, data augmentation, and population-based training on performance and robustness. The strategic properties of the agents are analyzed through the lens of game theory, providing insight into the gap between empirical performance and theoretical optimality.

2

Reinforcement Learning

Reinforcement Learning (RL) is a powerful paradigm within the field of machine learning that focuses on how intelligent agents can learn to make decisions by interacting with an environment. It is based on the concept of trial and error and the idea at the root resembles on the animal behaviours, included the human's one. That's the reason why one of the most innovative and studied field of Artificial Intelligence takes its origin from psychology observations. Before diving into the history and advanced topics of Reinforcement Learning, it is essential to first explore its basic concepts. This basic knowledge will provide a clearer understanding of how RL has evolved over time and the reasons behind its development.

Reinforcement Learning can be defined as an area of machine learning focused on how software agents should take actions in an environment to maximize cumulative reward. This definition condenses key concepts, providing basic knowledge and a general understanding of the algorithm. The first concept is the *agent*, representing the learner or decision-maker interacting with the *environment*, which refers to the world in which the agent operates and learns. In order to be able to interact with it, a representation of the current situation in the environment is needed and we call it *state*. The agent moves from state to state by interacting with the environment through *actions*, a decision or move that the agent can make and that generates a *rewards* which basically represent a feedback from the environment that evaluates the desirability of the action taken. Finally we have the *policy* which is the strategy that the agent uses to determine the next action based on the current state. All these elements interacting together form a loop that is visually showed in Figure 2.1.

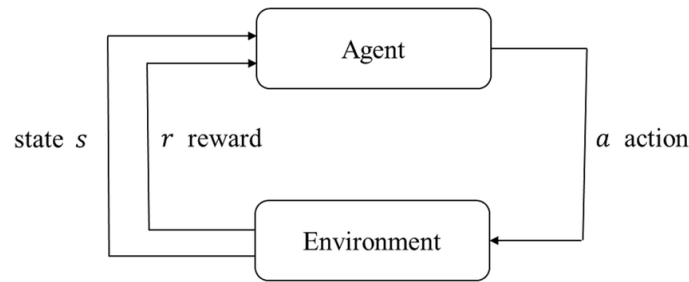


Figure 2.1: Typical interaction between state, action, reward, agent and environment

2.1 HISTORY OF REINFORCEMENT LEARNING

2.1.1 FOUNDATIONS OF LEARNING: ANIMAL BEHAVIORISM AND ITS IMPACT

Reinforcement Learning has its roots in behaviorist psychology, a branch of psychology that highlight the study of observable behaviors and their relationships with environmental stimuli, rather than focusing on internal cognitive processes. A key figure in the foundation of Reinforcement Learning was B.F. Skinner, an American psychologist, behaviorist and social philosopher best known for developing the theory of operant conditioning, a fundamental concept in behaviorism. He gave a comprehensive look at the principles of behavior analysis and at the theory of operant conditioning in his book called "Science and Human Behavior" published in 1953 [1]. The main insight of Skinner was that organisms (including humans) tend to repeat behaviors that are followed by positive consequences (rewards) and avoid behaviors that are followed by negative consequences (punishments). This process of learning through the consequences of one's actions is known as operant conditioning. In operant conditioning, a reinforcer is any event that increases the likelihood of a behavior being repeated. Positive reinforcers are pleasant events that are provided as a consequence of a behavior, while negative reinforcers are unpleasant events that are removed as a consequence. Skinner's experiments showed that by carefully controlling the reinforcement schedule - the pattern and timing of rewards and punishments - he could shape animal behavior in predictable ways. This principle is fundamental to Reinforcement Learning, where the agent learns to take actions that maximize the cumulative reward received from the environment. A concrete example of this theory is represented by Skinner's rat experiment. In this scenario, a rat is placed in a box equipped with a lever that, when pressed, releases food. Initially, the rat explores the box randomly, but once

it discovers that pressing the lever leads to a reward (the food), it starts pressing the lever more frequently. This behavior, reinforced by the reward, becomes a learned response, illustrating the process of learning through positive reinforcement.

Building on these foundational ideas of learning, the 1950s marked an important moment in the history of computer science and artificial intelligence. This decade saw the birth of "Artificial Intelligence" (AI) as a distinct field of study, with the term "Artificial Intelligence" itself being coined at the Dartmouth Conference in 1956. It was a time of great optimism and ambitious visions about the potential of computers to mimic human intelligence. During this period, researchers began to seriously consider how the principles of learning observed in biological systems, particularly in animals, could be applied to machines. This point of meeting of ideas between biology and computer science was driven by a few key factors like the advances in neuroscience that lead to significant progress in understanding the structure and function of neurons, but also the information theory that provided a mathematical basis for thinking about communication and signal processing, concepts that would prove crucial in developing learning algorithms. Indeed, in 1954, Marvin Minsky's PhD thesis, "Theory of Neural-Analog Reinforcement Systems and Its Application to the Brain Model Problem"[2], was one of the first works to explicitly connect ideas from animal learning to machine learning. Minsky's work was significant because it represented one of the first comprehensive attempts to bridge the gap between biological learning and machine learning. It showed how principles observed in animal behavior experiments could be formalized mathematically and implemented in artificial systems.

During the following years a great intuition has been possible thanks to Richard Bellman, a mathematician whose work on dynamic programming would be crucial in framing and solving sequential decision-making problems. Bellman's key idea was the principle of optimality, which he formalized in his 1957 book "Dynamic Programming" [3]. According to this principle, an optimal policy ensures that no matter the starting state or first action taken, the remaining actions must still form an optimal policy for the resulting state. This simple, but powerful idea allowed complex, multi-step decision problems to be broken down into simpler sub-problems, solving them recursively and combining the solutions to address the overall problem. The power of Bellman's work relies in its generality.

Dynamic programming provided a framework for optimizing decisions over time, applicable to a wide range of problems from resource allocation to trajectory planning. In the context of what would later become Reinforcement Learning, dynamic programming offered a way to compute optimal policies when the model of the environment was known. Bellman also intro-

duced the concept of the "value function," a key concept of modern RL. The value function captures the expected cumulative reward an agent can obtain from a given state, assuming it follows an optimal policy. This concept would later evolve into the state-value and action-value functions that are central to many RL algorithms.

Building upon Bellman's foundational work, Ronald Howard made a significant contribution in 1960 with his book "Dynamic Programming and Markov Processes" [4]. In his publication, Howard formally developed the framework of Markov Decision Processes (MDPs) and developed the idea of policy iteration which contributed significantly to the field of operations research and reinforcement learning by providing a systematic approach for solving MDPs.

2.1.2 FROM TEMPORAL DIFFERENCE LEARNING TO CONTEMPORARY REINFORCEMENT LEARNING

Moving forward, the 1970s saw a period of significant advancements. Robert Werbos's 1974 PhD thesis, "Beyond Regression: New Tools for Prediction and Analysis in the Behavioral Sciences"[5], was a revolutionary work that, in many ways, was ahead of its time. Werbos introduced several key concepts that would later become fundamental to both neural networks and Reinforcement Learning. In particular, it's worth to mention the Werbos's introduction of temporal difference learning concepts. Indeed, he proposed a way to learn predictions of future outcomes that didn't require waiting for the final result, instead updating predictions based on subsequent predictions. This idea of bootstrapping, using estimates to update estimates, is a core principle of temporal difference learning in RL. In the 1970s, significant advancements took place, particularly with Robert Werbos's groundbreaking 1974 PhD thesis, "Beyond Regression: New Tools for Prediction and Analysis in the Behavioral Sciences"[5]. This work, ahead of its time, introduced key concepts that later became fundamental to both neural networks and Reinforcement Learning. Notably, Werbos introduced the concept of temporal difference learning, proposing a method for making predictions of future outcomes without waiting for the final result. Instead, predictions were updated based on subsequent estimates, a process known as bootstrapping. This idea became a core principle of temporal difference learning in Reinforcement Learning. Going on, the 1980s saw a revival of interest in neural networks and learning algorithms, setting the stage for major advances in RL. In 1981, Andrew Barto, Richard Sutton, and Charles Anderson published their work on the actor-critic architecture [6]. This model was inspired by animal learning theories, particularly the role of dopamine in reward prediction, and neuroscience models of the basal ganglia, a part of the

brain involved in action selection and learning. This work was significant because it brought together several key ideas: temporal difference learning, policy gradients, and inspiration from neuroscience. It showed how these concepts could be combined into a powerful learning system capable of solving complex sequential decision problems. The culmination of this period came in 1988 with Richard Sutton's paper, "Learning to Predict by the Methods of Temporal Differences"[7]. This paper provided a clear, formal description of temporal difference (TD) learning algorithms and demonstrated their effectiveness in prediction problems. Sutton's TD learning formalization was a big step forward. It showed how predictions could be learned incrementally, without waiting for final outcomes. This made TD learning particularly well-suited for problems with long sequences of dependent events, a common scenario in RL tasks. The paper introduced the $TD(\lambda)$ algorithm, which uses a trace decay parameter λ to control the balance between bootstrapping (relying on current estimates) and Monte Carlo methods (waiting for actual outcomes). This flexibility allowed TD methods to be adapted to a wide range of problems. Sutton also provided theoretical analysis and empirical results showing that TD methods could learn more efficiently than previous approaches in many scenarios. This work set the foundation for many subsequent developments in RL, including Q-learning and SARSA in the following years.

The 1990s saw RL emerge as a distinct field within machine learning. In 1992, Christopher Watkins introduced Q-learning, an off-policy TD control algorithm. The following year, Satinder Singh and Richard Sutton introduced the SARSA algorithm, which serves as an on-policy alternative to Q-learning. Furthermore, Sutton and Barto in 1998 published a textbook, "Reinforcement Learning: An Introduction"[8], in which they provided a comprehensive treatment of the field and helped to establish its theoretical foundations. Even nowadays, this book is considered the definitive text for those new to RL.

This decade also saw the development of key algorithms that remain influential today. The early 2000s brought further refinements and extensions to RL theory and practice. Function approximation methods, particularly with neural networks, began to be more widely used to handle large state spaces and policy gradient methods, which optimize policies directly, also gained prominence during this period.

2.1.3 THE DEEP RL REVOLUTION

The integration of deep learning with RL was a huge shift in the field. Deep learning, with its ability to automatically learn hierarchical representations from raw data, addressed one of the

key limitations of traditional RL: the need for hand-crafted features or restrictive assumptions about the state space. In 2013, DeepMind (then a small startup, later acquired by Google) introduced the Deep Q-Network (DQN) algorithm. This breakthrough combined Q-learning, a well-established RL algorithm, with deep convolutional neural networks [9]. The result was a system that could learn to play Atari games directly from pixel inputs, achieving human-level performance on many games. What made DQN remarkable was its generality. The same architecture, with minimal adjustment, could learn to play a wide variety of games, from Pong to Breakout to Space Invaders. This demonstrated the power of combining RL's sequential decision-making framework with deep learning's representational capacity. The success of DQN generated intense interest in deep RL, leading to many innovations. Researchers developed techniques to stabilize training, improve sample efficiency, and scale to more complex environments. Variants like Double DQN and Prioritized Experience Replay further improved performance and stability.

While DQN was impressive, it was DeepMind's AlphaGo[10] that truly captured the world's attention. Go, an ancient board game with a vast state space and the challenge of evaluating positions, had long been considered a huge challenge for AI. Many experts believed it would take decades for AI to reach human-level performance. AlphaGo, which combined deep neural networks with Monte Carlo tree search, broke these expectations. In 2016, it defeated Lee Sedol, one of the world's top Go players, in a highly publicized match. This victory was a crucial moment, demonstrating the potential of deep RL to solve extremely complex problems. In the following years, DeepMind quickly followed up with AlphaGo Zero, which learned to play Go without any human data, using only self-play. This was then generalized to AlphaZero, which achieved superhuman performance in Go, chess, and shogi using the same algorithm[11]. Further advancements have been done by DeepMind with the introduction of MuZero[12], an algorithm that learns to model environments without needing to explicitly know the dynamics of the environment. MuZero extended these techniques to a wider range of tasks, showing remarkable performance in environments such as Atari games and the classic board games, illustrating the versatility and power of model-based reinforcement learning. These achievements were significant not just for their results, but for what they revealed about learning and intelligence. AlphaZero discovered new strategies and ways of playing that surprised human experts, suggesting that RL systems could not only match human performance but potentially surpass human knowledge in certain domains.

As we move into the 2020s and beyond, RL continues to be at the front line of AI research. Challenges such as sample efficiency, stability, and safe exploration remain active areas of inves-

tigation. The field is also fighting with broader questions about generalization, transfer learning, and the development of truly adaptive, general-purpose AI systems. The field is also facing wider questions about generalization, transfer learning, and the development of truly adaptive, general-purpose AI systems. In recent years, reinforcement learning has found a new and powerful application in the development and optimization of large language models (LLMs), marking another significant step forward in AI advancement. The development of ChatGPT by OpenAI in 2022 represented a major leap forward in natural language processing, powered in part by reinforcement learning techniques [13]. Specifically, ChatGPT utilizes a method called Reinforcement Learning from Human Feedback (RLHF), which combines RL with human preferences to fine-tune language models [14]. This approach allows the model to learn not just from static data, but from dynamic interactions and feedback, much like how RL agents learn to optimize their behavior in game environments. These developments highlight the adaptability and power of reinforcement learning, showing how techniques originally developed for game-playing AI can be adapted to revolutionize natural language processing and other domains of AI. As research continues, it's likely that RL will play an increasingly more important role in shaping the future of AI, contributing to the development of more general, adaptable, and capable artificial intelligence systems.

2.2 THE REINFORCEMENT LEARNING PROCESS

We have already explored the main concepts of Reinforcement Learning in Section 1.1 from an high level point of view. In order to dive into the core of RL, we need to introduce a rigorous approach to the subject and some mathematical formalisms for the elements seen up to now.

2.2.1 MARKOV DECISION PROCESSES (MDPs)

Markov Decision Processes (MDPs), already introduced in Section 2.1, constitute the base of modern Reinforcement Learning, providing a rigorous mathematical framework for modeling sequential decision-making under uncertainty. This formalism of MDP captures in a mathematical approach the core of the RL paradigm: an agent interacting with an environment over time, learning to make decisions that maximize cumulative reward. The strength of MDPs is represented by their ability to model complex, dynamic systems while keeping things manageable through the Markov property. This property allows the agent to focus solely on the current state and action, significantly reducing the computational complexity associated with

considering the entire history of past states. As a result, MDPs enable the development of efficient algorithms and facilitate the analysis of strategies, making the exploration of dynamic environments more achievable.

An MDP is formally defined as a tuple $(S, \mathcal{A}, P, R, \gamma)$, where each component plays a crucial role in describing the agent-environment interaction:

- S , the state space, represents all possible configurations of the environment. For example, in a chess game, S would include every possible arrangement of pieces on the board. The state $s_t \in S$ at time t encapsulates all information necessary for the agent to make informed decisions.
- $\mathcal{A}$ represents the action space, the set of all possible actions available to the agent. In our chess example, $\mathcal{A}$ would comprise all legal moves for the current player. The action $a_t \in \mathcal{A}$ at time t is the agent's choice, influencing the subsequent state and reward.
- P , the transition function, captures the dynamics of the environment. Formally, $P : S \times \mathcal{A} \rightarrow [0, 1]$ defines the probability of transitioning to a next state s' given that action a was taken in state s . Specifically, for each state s and action a , $P(s' \mid s, a)$ gives the probability of moving to state s' .
- R , the reward function, quantifies the desirability of state-action pairs. It is expressed as $R : S \times \mathcal{A} \rightarrow \mathbb{R}$, specifying the immediate reward received when action a is taken in state s . The reward function is the primary mechanism for addressing the agent's objectives, guiding it towards desired behaviors.
- γ , the discount factor, balances the agent's consideration of immediate versus future rewards, with $\gamma \in [0, 1]$.

To perform well in its task, the agent must not just focus on maximizing immediate rewards from each action. Rather, the general goal of an RL agent is to maximize the *expected cumulative reward* over time, also known as the *Return*.

Given a reward R_t received at time step t , the Return G_t , or cumulative reward from time step t onwards, can be mathematically formalized as:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad (2.1)$$

As seen before, the term γ is the *discount factor* that plays a crucial role in balancing immediate versus future rewards. It can be a value between 0 and 1, where values closer to 1 indicate a stronger attention on long-term rewards, while values closer to 0 cause the agent to prioritize

immediate rewards. The choice of γ significantly influences the agent's behavior and the performance of the learned policy, as it determines how much weight is given to future rewards compared to immediate ones.

$$P(s_{t+1} \mid s_t, a_t) = P(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0, a_0) \quad (2.2)$$

This property implies that the current state s_t contains all the necessary information to make decisions about future actions. Then, an agent only needs to consider the current state to determine the optimal action, rather than needing to remember the entire history of previous states and actions. This greatly simplifies the problem of making decisions over time, as it reduces the complexity of planning and evaluating future actions. While this assumption may seem restrictive, many complex systems can be modeled as MDPs by carefully designing the state space to include all relevant historical information. By designing the state space to incorporate not only the current conditions but also essential elements of past events, the property can be maintained. Thanks to this approach MDPs can effectively model and address complex sequential decision-making problems.

2.2.2 VALUE FUNCTIONS AND BELLMAN EQUATIONS

In the framework of Markov Decision Processes, a policy π defines the agent's behavior by mapping states $s \in S$ to actions $a \in A$, or, in the case of stochastic policies, to a probability distribution over actions, $P(A)$. Formally, this is represented as $\pi : S \rightarrow A$ or $\pi : S \rightarrow P(A)$. As previously discussed, the primary goal of reinforcement learning is to discover an optimal policy π^* that maximizes the expected cumulative discounted reward over time. This objective is mathematically formulated as:

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid \pi \right] \quad (2.3)$$

where γ represents the discount factor, $R(s_t, a_t, s_{t+1})$ denotes the reward obtained by taking action a_t in state s_t , and $\mathbb{E}[\cdot \mid \pi]$ means the expectation over the policy π .

To facilitate the search for the optimal policy, we use two fundamental functions: the *state-value function* $V_{\pi}(s)$ and the *action-value function* $Q_{\pi}(s, a)$.

The *state-value function* $V_{\pi}(s)$ represents the expected return when an agent starts in state s and follows the policy π thereafter. It reflects the long-term value of being in state s under policy π . This function is useful because it evaluates the desirability of being in a given state while

following a specific policy and allows to make decisions based on anticipated future rewards. Mathematically, the state-value function $V_\pi(s)$ is expressed as:

$$V_\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s] \quad (2.4)$$

Expanding this, we have:

$$V_\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid s_0 = s \right] \quad (2.5)$$

Here, the expectation is taken over all possible sequences of states and actions that come after from starting at state s and following policy π , providing a measure of how beneficial it is to be in state s .

On the other hand, the *action-value function* $Q_\pi(s, a)$ evaluates the expected return when taking a specific action a in state s and then following policy π . This function is valuable as it provides insight into the immediate value of a particular action in a given state, considering both the immediate reward and the future rewards obtained by continuing with policy π . Mathematically, the action-value function is:

$$Q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a] \quad (2.6)$$

Expanding this, the action-value function is:

$$Q_\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t, s_{t+1}) \mid s_0 = s, a_0 = a \right] \quad (2.7)$$

This expression indicates the expected cumulative discounted reward when the agent starts in state s , takes action a , and then follows policy π . It helps in determining which actions are preferable in different states, therefore helping in decision-making to maximize cumulative rewards.

The relationship between these functions is represented in the Bellman equations, which are foundational to many reinforcement learning algorithms. The Bellman equation for the state-value function $V_\pi(s)$ is:

$$V_{\pi}(s) = \sum_a \pi(a | s) \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V_{\pi}(s')] \quad (2.8)$$

where $\pi(a | s)$ is the probability of taking action a in state s , and $P(s' | s, a)$ is the probability of transitioning to state s' from state s given action a .

Instead, for the action-value function $Q_{\pi}(s, a)$, the Bellman equation is:

$$Q_{\pi}(s, a) = \sum_{s'} P(s' | s, a) \left[R(s, a, s') + \gamma \sum_{a'} \pi(a' | s') Q_{\pi}(s', a') \right] \quad (2.9)$$

These equations provide a recursive structure for computing the value functions and are fundamental to dynamic programming approaches in reinforcement learning. By solving these Bellman equations, one can systematically derive optimal policies that maximize the expected return. This recursive nature simplifies the problem of finding optimal policies by breaking it down into smaller subproblems, making it manageable and efficient to compute optimal strategies for complex decision-making processes.

2.2.3 OPTIMAL POLICIES AND THE PATH TO MAXIMIZATION

As we have just seen, an optimal policy π^* is one that maximizes the expected cumulative discounted reward from any given state. It's important to mention that all optimal policies obtain the same maximum value function. This means that while multiple optimal policies can exist, they all result in the same expected returns.

Now, we introduce two key concepts: the *optimal state-value function* $V^*(s)$ and the *optimal action-value function* $Q^*(s, a)$. These functions represent the maximum expected return obtainable from each state or state-action pair, respectively, when following any optimal policy. The Bellman optimality equations for $V^*(s)$ and $Q^*(s, a)$ provide a recursive definition for the optimal value functions:

$$V^*(s) = \max_a \left[R(s, a) + \gamma \sum_{s'} P(s' | s, a) V^*(s') \right] \quad (2.10)$$

$$Q^*(s, a) = R(s, a) + \gamma \sum_{s'} P(s' | s, a) \max_{a'} Q^*(s', a') \quad (2.11)$$

These equations express the optimal value functions in terms of the best immediate reward plus the discounted future reward obtained from the optimal actions. They are fundamental

in determining the optimal policy as they describe the necessary conditions for a policy to be optimal and they play a crucial role in finding optimal policies by breaking down a complex problem into simpler, more manageable parts.

Two primary methods that leverage the Bellman optimality equations to converge towards optimal policies are:

- **Policy Iteration:** Alternates between policy evaluation and policy improvement.
- **Value Iteration:** Updates value functions iteratively until convergence to the optimal value functions.

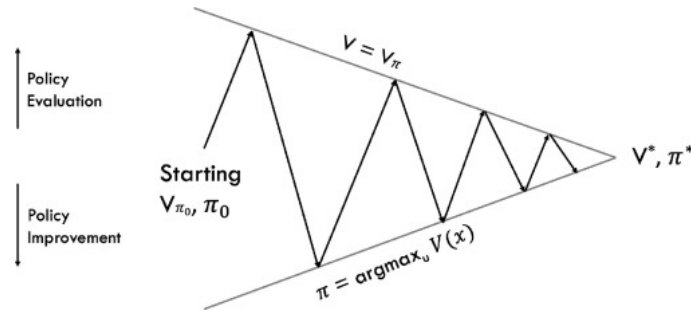


Figure 2.2: Policy Iteration schema

In the policy evaluation phase, the current policy is assessed by computing the state-value function $V_{\pi}(s)$ using the Bellman equation until the values stabilize. During the policy improvement phase, the policy is updated by selecting actions that maximize the expected return based on the evaluated value function. The Policy Iteration process continues, alternating between evaluation and improvement, until the policy converges to the optimal policy π^* as shown at Figure 2.2. On the other hand, Value Iteration combines policy evaluation and policy improvement into a single step by updating the value function $V(s)$ based on the Bellman optimality equation: $V^*(s) = \max_a [R(s, a) + \gamma \sum_{s'} P(s' | s, a) V^*(s')]$. This process continues until the value function converges to $V^*(s)$, at that point the optimal policy π^* can be derived by selecting the action that maximizes the expected return for each state.

Therefore, we can say that finding the optimal policy π^* involves a process of iterative updates and refinements of both policies and value functions, exploiting the recursive structure of the Bellman equations. By repeatedly evaluating and improving the policy, this approach systematically converges on strategies that increasingly enhance the expected returns. The iterative nature of this process allows the agent to learn from previous iterations, adjusting its decisions

based on the evolving estimations of state and action values. This progression is not simply about local adjustments but represents a continuous convergence towards optimality, where each iteration fine-tunes the policy to exploit the balance between immediate rewards and long-term gains. Through dynamic programming methods such as Policy Iteration and Value Iteration, the recursive updates provide a framework for the agent to explore and exploit the state space efficiently, leading to the discovery of an optimal policy that maximizes cumulative rewards over time.

2.2.4 EXPLORATION VS. EXPLOITATION DILEMMA

The exploration vs. exploitation dilemma represents a popular challenge in reinforcement learning and decision-making processes. This dilemma is about the tension between two objectives: exploring new actions to discover potentially higher rewards (exploration) and leveraging known actions that have previously obtained high rewards (exploitation). In the context of reinforcement learning, exploration involves an agent proceeding into unknown territory, trying out new actions to obtain information about the environment and potential rewards. On the other side, exploitation refers to the agent's tendency to take advantage of its current knowledge, selecting actions that it believes will maximize immediate rewards based on past experiences.

Balancing exploration and exploitation is crucial for discovering the optimal policy in an environment. Without sufficient exploration, an agent may converge prematurely to a sub-optimal policy, missing out on potentially better strategies. Therefore, several methods have been developed to address the exploration aspect of the dilemma:

- **Epsilon-Greedy (ϵ -greedy):** This simple but effective strategy involves selecting a random action with probability ϵ and the best-known action with probability $1 - \epsilon$. The value of ϵ can be adjusted to control the exploration rate, often starting high and decreasing over time.
- **Softmax Action Selection:** This method assigns probabilities to actions based on their estimated values, typically using a Boltzmann distribution. This approach allows for a smarter exploration strategy, where actions with higher estimated values have a higher probability of being selected, but lower-valued actions still have a chance.
- **Upper Confidence Bound (UCB):** UCB algorithms select actions based on an optimistic estimate of their potential reward, considering both the estimated value and the uncertainty associated with that estimate. This strategy effectively balances exploration and exploitation by favoring actions with high uncertainty or high estimated value.

The exploration-exploitation balance plays a crucial role in determining the efficiency of learning in reinforcement learning. When an agent goes in for excessive exploration, it can slow down the convergence to an optimal policy, while insufficient exploration may result in premature convergence to a suboptimal solution. Finding the right balance not only enhances convergence speed but also enables the agent to adapt rapidly to changes in the environment, which is essential for maintaining performance in non-stationary settings. Moreover, effective balancing significantly improves sample efficiency, allowing the agent to obtain optimal behaviors with fewer interactions. Eventually, a well-balanced approach encourages the development of more robust policies that perform effectively across various environmental conditions, improving the agent's overall effectiveness and adaptability. However, it's not just about discovering new states. Indeed, the exploration-exploitation dilemma presents a fundamental trade-off between short-term and long-term rewards. Exploitation typically maximizes immediate rewards, which can be advantageous in situations where rapid performance improvements are essential. However, this short-term focus may miss opportunities for exploration that could discover strategies leading to higher long-term rewards. In dynamic, non-stationary environments, the capacity to sacrifice short-term gains in favor of long-term adaptability becomes crucial. This underlines the importance of maintaining a degree of exploration even in the later stages of learning, guaranteeing that the agent remains capable of adjusting to new evolving conditions while optimizing its overall performance. In conclusion, the exploration vs. exploitation dilemma remains a central challenge, even in advanced topics, with significant implications for the efficiency, adaptability, and general performance of learning agents.

2.2.5 KEY METHODS: Q-LEARNING, SARSA, AND TEMPORAL DIFFERENCE LEARNING

In Reinforcement Learning, choosing the right learning method is crucial for an agent to succeed. As we have seen in the previous sections, by learning both V and Q , the agent can better understand the value of being in different states and the potential outcomes of its actions, guiding it into optimal decision-making.

This section introduces two main types of methods: *on-policy* and *off-policy*. On-policy methods, like SARSA, learn from the actions the agent actually takes, while off-policy methods, such as Q-Learning, learn about the best actions regardless of the agent's current choices.

Q-LEARNING: OFF-POLICY LEARNING

Q-Learning is one of the most popular and influential algorithms in reinforcement learning. As an off-policy method, Q-Learning learns the optimal action-value function independently of the agent's current policy. This characteristic allows Q-Learning to estimate optimal future rewards even while the agent may be following a different policy. Q-Learning operates by maintaining a Q-table (or Q-function) that estimates the expected cumulative reward for each state-action pair. The algorithm updates these estimates iteratively based on the rewards received and the estimated future values. Here is the Q-Learning algorithm:

Algorithm 2.1 Q-Learning Algorithm

```
Initialize  $Q(s, a)$  arbitrarily for all state-action pairs
Define learning rate  $\alpha$  and discount factor  $\gamma$ 
for each episode
    Initialize starting state  $s$ 
    while the episode is not done
        Choose action  $a$  using an exploration strategy (e.g.,  $\epsilon$ -greedy)
        Take action  $a$ , observe reward  $r$  and next state  $s'$ 
        Update Q-value:
```

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

```
        Set  $s \leftarrow s'$  (move to next state)
    end while
end for
```

The core of Q-Learning is its update rule:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \quad (2.12)$$

This update rule is a form of temporal-difference learning, where the algorithm updates its estimate based on the immediate reward and the best estimated future value. The learning rate α determines how much the Q-value is updated based on the new information. A higher α results in faster learning but may lead to instability, while a lower α leads to slower but potentially more stable learning. Q-Learning's off-policy nature is evident in the max operator in its update rule. By taking the maximum over all possible next actions, Q-Learning updates its estimates towards

the optimal policy, independently from the action actually taken by the agent. This property allows Q-Learning to learn about the optimal policy even while following an exploratory policy, making it particularly suitable for scenarios where exploration is necessary or desirable.

SARSA: ON-POLICY LEARNING

SARSA (State-Action-Reward-State-Action) is a popular algorithm in reinforcement learning that represent on-policy learning. Unlike off-policy methods such as Q-Learning, which learn the optimal action-value function independently of the agent's current policy, SARSA updates its estimates based on the actions taken by the agent. This characteristic means that SARSA evaluates the effectiveness of the strategy the agent is currently using, allowing it to adapt its learning based on the agent's actual behavior.

SARSA maintains a Q-table (or Q-function), as Q-Learning, that estimates the expected cumulative reward for each state-action pair. The algorithm updates these estimates iteratively based on the rewards received and the actions taken by the agent. The basic structure of the SARSA algorithm is as follows:

Algorithm 2.2 SARSA Algorithm

```

1: Initialize  $Q(s, a)$  arbitrarily for all state-action pairs
2: Define learning rate  $\alpha$  and discount factor  $\gamma$ 
3: for each episode
4:   Initialize starting state  $s$ 
5:   Choose action  $a$  using an exploration strategy (e.g.,  $\epsilon$ -greedy)
6:   while the episode is not done
7:     Take action  $a$ , observe reward  $r$  and next state  $s'$ 
8:     Choose next action  $a'$  using the same exploration strategy
9:     Update Q-value:

```

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)]$$

```

10:   Set  $s \leftarrow s'$  and  $a \leftarrow a'$  (move to next state and action)
11: end while
12: end for

```

Again, the most relevant part of the algorithm is its update rule:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)] \quad (2.13)$$

This update rule reflects the on-policy nature of SARSA, as it uses the value of the action a' chosen in the next state s' to update the current Q-value. In contrast, Q-Learning uses the maximum estimated Q-value over all possible actions in the next state, making it an off-policy method. The learning rate α plays a crucial role in determining how quickly the Q-values are updated in response to new information. Similar to Q-Learning, a higher α leads to faster learning but may introduce instability, while a lower α promotes stability at the cost of slower convergence.

SARSA's on-policy approach means that it is sensitive to the chosen exploration strategy. As the agent explores the environment, it updates its action-value estimates based on the actions it actually takes. This property can take to more conservative learning, making SARSA particularly effective in environments where following a specific policy is crucial, as it allows the agent to improve its strategy in line with the exploration behavior.

Figure 2.3 provides a visual representation of Q-Learning and SARSA. Black dots represent states, white circles represent actions, and arrows indicate state transitions based on chosen actions. The schema illustrates how the two algorithms update their value functions. Q-learning updates its value function based on the maximum expected future reward, while SARSA updates it based on the expected future reward of the action actually taken (following the state-action pair).

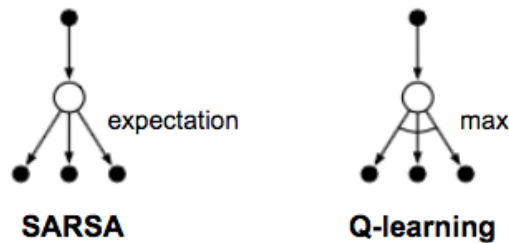


Figure 2.3: A Visual Comparison of SARSA and Q-Learning Algorithms

TEMPORAL LEARNING: MONTE CARLO METHODS AND TD(λ)

Monte Carlo methods and Temporal Difference (TD) learning are two fundamental approaches in reinforcement learning that differ in their strategies for estimating value functions. While both methods aim to improve the agent's understanding of its environment through experience, they do so using distinct patterns.

Monte Carlo methods rely on complete episodes to evaluate the value of states or actions. By

averaging the returns (cumulative rewards) received after multiple visits to a state or action, these methods provide an unbiased estimate of the expected reward. However, Monte Carlo approaches require waiting until the end of an episode to perform updates, which can lead to inefficiencies, particularly in environments where episodes are particularly long. The basic structure of the Monte Carlo algorithm is as follows:

Algorithm 2.3 Monte Carlo Algorithm

```

1: Initialize  $V(s)$  arbitrarily for all states
2: for each episode
3:   Generate an episode  $s_0, a_0, r_1, s_1, a_1, r_2, \dots, s_T$ 
4:   for each state  $s$  in the episode
5:     Compute the return  $G$  for state  $s$ 
6:     Update value estimate:
7:      $V(s) \leftarrow V(s) + \alpha (G - V(s))$ 
8:   end for
9: end for

```

In contrast, $TD(\lambda)$ extends the TD learning framework by integrating ideas from both Monte Carlo methods and traditional TD learning. It allows the agent to update value estimates based on multiple future rewards, weighted by how far into the future those rewards happen. This approach is more flexible and can balance between immediate and long-term rewards. The update rule for $TD(\lambda)$ can be represented as:

$$V(s) \leftarrow V(s) + \alpha [G_t - V(s)] \quad (2.14)$$

where G_t is the n -step return, which considers rewards received over multiple steps, weighted by the parameter λ . The basic structure of the $TD(\lambda)$ algorithm is as follows:

Algorithm 2.4 TD(λ) Algorithm

```
1: Initialize  $V(s)$  arbitrarily for all states
2: for each episode
3:   Initialize starting state  $s$ 
4:   Initialize  $E(s) \leftarrow 0$  for all states
5:   while the episode is not done
6:     Take action  $a$ , observe reward  $r$  and next state  $s'$ 
7:     Compute TD error:  $\delta \leftarrow r + \gamma V(s') - V(s)$ 
8:     Update eligibility traces:  $E(s) \leftarrow \gamma \lambda E(s) + 1$ 
9:     for each state  $s$ 
10:      Update value estimate:
                                     
$$V(s) \leftarrow V(s) + \alpha \delta E(s)$$

11:    end for
12:    Set  $s \leftarrow s'$ 
13:  end while
14: end for
```

Figure 2.4 visually shows the Temporal Difference (TD) learning algorithm, using the same notation as Figure 2.3. Black dots represent states, white circles represent actions, and arrows indicate state transitions based on chosen actions. By changing the parameter λ in the TD(λ) algorithm, we can adjust the future reward prediction. This allows us to interpolate between one-step TD learning and Monte Carlo methods, effectively controlling how far into the future the algorithm looks to update its estimates.

Together, Monte Carlo methods and TD(λ) highlight the trade-offs between the completeness of episode-based learning and the efficiency of incremental updates, each offering unique advantages depending on the problem context. Understanding these methods is essential for developing effective reinforcement learning algorithms.

2.2.6 VALUE FUNCTION APPROXIMATION

Value function approximation is a basic technique in modern reinforcement learning. It's able to give an answer to the challenge of estimating and representing value functions in complex, high-dimensional environments. As RL applications have expanded reaching increasingly sophisticated domains, from game playing to robotic control and beyond, the need for efficient

$$Q(s, a) \approx \mathbf{w}^T \boldsymbol{\varphi}(s, a)$$

Where $\mathbf{w}$ is a weight vector and $\boldsymbol{\varphi}(s)$ or $\boldsymbol{\varphi}(s, a)$ is a feature vector representing the state s or state-action pair (s, a) . The goal of learning is to find the optimal weight vector $\mathbf{w}$ that minimizes the difference between the true value function and its approximation. Linear models are easy to understand, implement, and debug, however, they may fail to capture complex, non-linear relationships in the value function. That's why non-linear methods have been introduced. Non-linear function approximation methods, particularly those based on neural networks, have gained significant relevance in recent years thanks to their ability of capturing complex patterns in high-dimensional data. These methods can represent the value function as a complex, non-linear transformation of the input state or state-action pair. Neural networks, especially deep neural networks, are the most common form of non-linear function approximators in modern RL. A typical neural network representation might look like:

$$V(s) \approx f(s; \theta)$$

where f is a neural network with parameters θ , taking state s as input.

THE REINFORCE ALGORITHM

The REINFORCE algorithm, also known as the policy gradient method, is a Monte Carlo policy-based approach that directly optimizes the policy without explicitly maintaining a value function. REINFORCE updates the policy parameters to maximize the expected return, using the likelihood ratio trick to estimate the gradient of the expected return with respect to the policy parameters. The algorithm iteratively improves the policy by following this gradient. The algorithm can be summarized as follows:

The mathematical formulation of the REINFORCE algorithm is derived from the policy gradient theorem, which provides the gradient of the expected return $J(\theta)$ with respect to the policy parameters θ :

$$\nabla_{\theta} J(\theta) = \mathbb{E}[\nabla_{\theta} \log \pi(a|s; \theta) Q^{\pi}(s, a)]$$

where $\pi(a|s; \theta)$ represents the policy, and $Q^{\pi}(s, a)$ represent the action-value function under policy π .

Algorithm 2.5 REINFORCE Algorithm

```
1: Initialize policy parameters  $\theta$ 
2: for each episode
3:   Initialize empty lists of states  $S$ , actions  $A$ , and rewards  $R$ 
4:   while the episode is not done
5:     Select action  $a$  based on policy  $\pi(a|s; \theta)$ 
6:     Observe reward  $r$  and next state  $s'$ 
7:     Append  $s$ ,  $a$ , and  $r$  to  $S$ ,  $A$ , and  $R$ 
8:     Set  $s \leftarrow s'$ 
9:   end while
10:  Compute returns  $G_t$  for each time step  $t$ 
11:  for each time step  $t$ 
12:    Update policy parameters:
```

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$$

where

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} \log \pi(a|s; \theta) G_t$$

```
13:  end for
14: end for
```

ACTOR CRITIC ALGORITHM

Actor-Critic methods combine value function approximation with policy optimization, finding a solution to some of the limitations of pure policy gradient methods. The architecture consists of two main components: the actor and the critic. The actor represents and updates the policy, while the critic estimates the value function, which is used to guide the actor's updates. The key concepts of Actor-Critic methods include the following: the critic reduces the variance of policy updates by providing a baseline, and the actor uses the critic's value estimates to update the policy more efficiently. The mathematical formulation for the actor's update typically follows a form similar to:

The Actor-Critic approach provides several advantages over traditional methods. By maintaining separate structures for policy and value function estimation, it allows for more stable learning. The critic's feedback helps mitigate the high variance associated with pure policy gradient methods, allowing more efficient exploration of the action space. This interaction between the actor and the critic encourage a more effective learning process, often resulting in faster convergence and improved performance.

Algorithm 2.6 Actor-Critic Algorithm

```
1: Initialize policy parameters  $\theta$  and value function parameters  $w$ 
2: for each episode
3:   Initialize starting state  $s$ 
4:   while the episode is not done
5:     Select action  $a$  based on policy  $\pi(a|s; \theta)$ 
6:     Observe reward  $r$  and next state  $s'$ 
7:     Compute TD error:

$$\delta \leftarrow r + \gamma V(s'; w) - V(s; w)$$

8:     Update value function parameters:

$$w \leftarrow w + \alpha_v \delta \nabla_w V(s; w)$$

9:     Update policy parameters:

$$\theta \leftarrow \theta + \alpha_a \delta \nabla_\theta \log \pi(a|s; \theta)$$

10:    Set  $s \leftarrow s'$  (move to next state)
11:  end while
12: end for
```

3

Ultimate Tic Tac Toe

Ultimate Tic Tac Toe (UTTT) is a complex variant of the classic Tic Tac Toe game that has recently gained attention in the fields of game theory and artificial intelligence. This chapter explores the complexities of UTTT, its rules, strategic challenges, and its relevance in various research domains. The origin of UTTT comes from the early 2000s, although the exact creator remains unclear. It emerged as a natural extension of the traditional Tic Tac Toe, addressing the simplicity and quick resolution of its predecessor. While classic Tic Tac Toe is often solved by young children and can result in frequent draws between experienced players, UTTT introduces a layer of complexity that challenges even advanced strategists and computers. Compared to traditional Tic Tac Toe, UTTT offers a significantly expanded game space and decision tree. This increased complexity not only makes the game more engaging for human players but also presents a fertile ground for studying decision-making processes, strategic planning, and the development of artificial intelligence algorithms. The game's popularity has grown during the years, particularly in academic and competitive gaming circles. Its relevance in game theory studies takes its origin from its unique structure, which combines local and global strategic considerations, creating a rich environment for analyzing decision-making under particular conditions.

3.0.1 GAME RULES

There exists many variants of UTTT that can be more or less challenging. In our experiment, the game is played on a 3×3 grid of 3×3 Tic Tac Toe boards, resulting in a 9×9 global board (Figure 3.1). Each of the nine local 3×3 boards represents a cell in the larger global board.

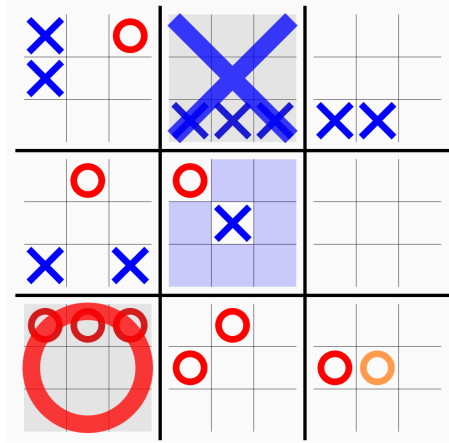


Figure 3.1: Board of Ultimate Tic Tac Toe

The game starts with Player X making a move in any cell of any local board. The crucial rule that adds complexity to the game is that the position of the last move in a local board determines the local board where the next player must play. If Player X places their mark in the top-right cell of a local board, Player O must then make their move in the corresponding top-right local board of the global grid (as shown in the example at Figure 3.2). However, if the required local board is already full or has been won, the next player can choose to play in any open cell on any available local board that has not been won or drawn. This rule creates a strategic link between local and global gameplay, forcing players to balance immediate tactical decisions with long-term strategic planning. The winning condition in UTTT operates on two levels:

- **Local Board:** A player wins a local board by achieving three of their marks in a row, column, or diagonal within that board.
- **Global Board:** The game is won by claiming three local boards in a row, column, or diagonal on the global board.

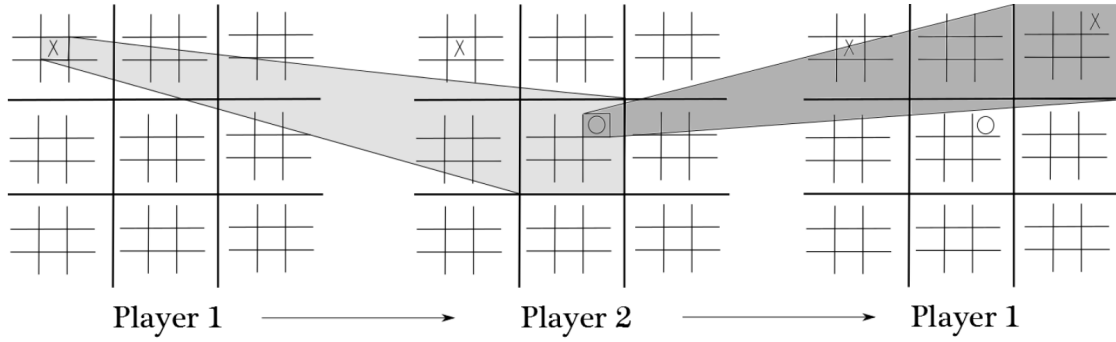


Figure 3.2: Visualization of UTTT's game dynamics

The complexity of UTTT far exceeds that of traditional Tic Tac Toe. While classic Tic Tac Toe has a game tree complexity of approximately 255,168 possible playable games [15], UTTT presents a even more complex game tree representing a significant computational challenge, which makes it difficult to fully solve.

Indeed, there are 81 cells for a 3x3 grid of 3x3 tic-tac-toe games, as shown in Figure 3.1. Each cell can have 3 possible values (empty, X or O), the number of possible states goes from 3^9 for regular tic-tac-toe to 3^{81} , which is definitely more than the number of stars in the entire universe. Actually, all the states are not reachable due to the rules of the game, but this still gives an idea of the vastness of the game. This increase in complexity places UTTT closer to games like chess in terms of strategic depth and computational challenge. The branching factor of UTTT varies throughout the game but is significantly higher than classic Tic Tac Toe.

3.0.2 RELATED WORKS

The game of Ultimate Tic-Tac-Toe has not been extensively explored in the literature compared to other games. Several studies have been conducted, but the game's numerous variants complicate comparisons with research focused on its different rule sets. Giving example, one notable contribution is a study regarding an optimal winning strategy for UTTT, where researchers demonstrated that victory can be achieved in at most 43 moves by an optimal agent, offering theoretical bases and formal proofs regarding the game's structure and complexity [16]. Unfortunately, the rules used in that study were slightly different from those applied in our research, so we cannot use those demonstrations in our case.

While several efforts have aimed at developing agents with human-like performance, there is a lack of significant, large-scale research conducted by professionals in the field. A relevant part of these works involves adapting AlphaZero-inspired approaches, where Monte Carlo Tree Search

(MCTS) is combined with a policy-value neural network. Projects such as uttt.ai [17] have explored this methodology, using self-play to teach agents effective UTTT strategies. Another recent study conducted by Furman University [18] tested multiple agents to solve the game; however, the results did not demonstrate extraordinary performance. Beyond these, most work in the domain consists of smaller-scale projects by students or independent researchers, leaving the area underexplored in terms of major academic studies.

4

The algorithms behind Ultimate Tic Tac Toe

This chapter explores from a theoretical point of view the algorithms that form the basis of this research on Ultimate Tic Tac Toe. Each algorithm contributes with distinct strengths and methodologies, facilitating strategic decision-making in this complex game.

4.1 DEEP Q-NETWORK

Deep Q-Network (DQN) is a leading algorithm in deep reinforcement learning that has been presented for the first time in 2013 by DeepMind [9]. The algorithm builds upon the principles of Q-learning, a model-free reinforcement learning technique already presented in Section 2.2.5. As a brief reminder, Q-learning operates by maintaining a Q-table (also called Q-function) that estimates the expected cumulative reward for each state-action pair. The key innovation in DQN is the use of a neural network to approximate the Q-function, which estimates the quality of actions in different states. So, at its base, DQN utilizes Q-function approximation thanks to its neural network. In the context of UT'TT, the DQN takes the game state representing the board as input and outputs Q-values for all possible actions.

The algorithm of DQN is exactly the same proposed for the classic Q-learning with the only difference on the update rule:

$$Q(s_t, a_t; \theta) \leftarrow Q(s_t, a_t; \theta) + \alpha \left[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a'; \theta) - Q(s_t, a_t; \theta) \right] \quad (4.1)$$

The parameter θ refers to the weights and biases of the neural network. These parameters are updated through backpropagation during training, aiming to minimize the difference between the Q-values predicted by the network and the target Q-values. Unlike Q-learning, where Q-values are stored in a table and updated directly, in DQN, the neural network generalizes across states, allowing it to handle large or continuous state spaces, such as those found in games like Ultimate Tic Tac Toe.

4.2 DOUBLE Q-LEARNING

Double Q-Learning is an extension of the standard Q-learning algorithm, introduced to address a common issue known as *overestimation bias*, which might happen in scenarios where the maximum overestimated action values are used for updating Q-values. This problem is particularly significant in environments with noisy rewards or when the Q-function is approximated by a neural network, as in Deep Q-Networks (DQN). Double Q-Learning was introduced by Hado van Hasselt with his team from Google DeepMind in 2010 [19] and later integrated into deep reinforcement learning to form Double DQN (DDQN).

The core idea of Double Q-Learning is to divide the action selection from the Q-value estimation to reduce overoptimistic value estimates. This is achieved by maintaining two separate Q-functions, denoted as Q_A and Q_B , each updated independently. During training, the action that maximizes the Q-value is chosen using one set of weights, while the value of that action is estimated using the other set. This division mitigates the bias in the Q-value update.

Formally, the update rule in Double Q-Learning can be expressed as follows:

$$Q_A(s_t, a_t) \leftarrow Q_A(s_t, a_t) + \alpha \left[r_{t+1} + \gamma Q_B(s_{t+1}, \arg \max_{a'} Q_A(s_{t+1}, a')) - Q_A(s_t, a_t) \right] \quad (4.2)$$

In this equation, the action selection $\arg \max_{a'} Q_A(s_{t+1}, a')$ is determined using Q_A , but the Q-value for the chosen action is evaluated using Q_B , and vice versa when updating Q_B .

When applied to deep reinforcement learning, this approach is referred to as Double DQN (DDQN), where two neural networks are used to approximate Q_A and Q_B . In practice, however, only one network is maintained, with a second network serving as a *target network* that is updated less frequently, following the same update mechanism as DQN. This modification significantly reduces overestimation errors, improving the stability and performance of the learning process in complex environments like Ultimate Tic Tac Toe.

The Double Q-Learning algorithm can be summarized as follows:

Algorithm 4.1 Double Q-Learning Algorithm

- 1: Initialize $Q_A(s, a)$ and $Q_B(s, a)$ arbitrarily for all state-action pairs
- 2: Define learning rate α and discount factor γ
- 3: **for** each episode
- 4: Initialize starting state s
- 5: **while** the episode is not done
- 6: Choose action a using an exploration strategy (e.g., ϵ -greedy)
- 7: Take action a , observe reward r and next state s'
- 8: With probability p , update Q_A :

$$Q_A(s, a) \leftarrow Q_A(s, a) + \alpha \left[r + \gamma Q_B(s', \arg \max_{a'} Q_A(s', a')) - Q_A(s, a) \right]$$

- 9: Otherwise, update Q_B :

$$Q_B(s, a) \leftarrow Q_B(s, a) + \alpha \left[r + \gamma Q_A(s', \arg \max_{a'} Q_B(s', a')) - Q_B(s, a) \right]$$

- 10: Set $s \leftarrow s'$ (move to next state)
 - 11: **end while**
 - 12: **end for**=0
-

Figure 4.1 shows how the update of the Q-function works with a DDQN and might help to understand the concept.

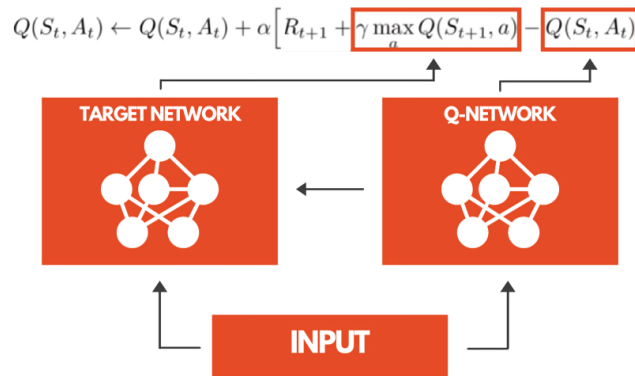


Figure 4.1: Visualization of Double DQN's update rule functioning

4.3 ACTOR-CRITIC

The Actor-Critic method represents a fundamental approach to reinforcement learning that combines policy optimization with value function estimation, as discussed in Section 2.2.6.

In the classic Actor-Critic framework, two main components interact continuously: the actor and the critic. The actor is responsible for proposing actions based on the current policy, while the critic evaluates these actions by estimating the expected returns. This dual mechanism allows for more stable learning dynamics and enhanced performance. In this implementation, neural networks are utilized to approximate both the actor's policy and the critic's value function, allowing the method to manage complex state and action spaces effectively. The Actor-Critic method contrasts with value-based approaches such as Deep Q-Networks (DQN) and Double DQN (DDQN). While DQN and DDQN focus just on estimating action values using a Q-function, the classic Actor-Critic method maintains separate structures for policy and value function estimation. DQN employs a neural network to approximate Q-values, while the actor in the classic Actor-Critic method updates the policy based on feedback from the critic's value estimates. This separation allows the Actor-Critic method to reduce variance in policy updates and achieve more efficient exploration of the action space.

The actor's updates in the classic method are typically guided by the temporal difference (TD) error, which quantifies the difference between the expected value and the observed reward. By leveraging this feedback, the actor can adapt its policy to maximize expected returns, benefiting from the critic's more stable value estimates. Algorithm 2.6 represents the general Actor-Critic algorithm, and like DQN and other methods, its most crucial aspect is the update rule, which can be expressed as:

$$\theta \leftarrow \theta + \alpha_a \delta \nabla_{\theta} \log \pi(a|s; \theta) \quad (4.3)$$

where δ is the TD error, defined as:

$$\delta \leftarrow r + \gamma V(s'; w) - V(s; w) \quad (4.4)$$

The A2C structure might seem complicated at first sight, but it's able to combine the strengths of both policy-based and value-based approaches, allowing for more stable learning and improved sample efficiency. In Figure 4.2, a schema representing the structure of an Actor-Critic system is shown, which includes all the elements that have been presented up to now.

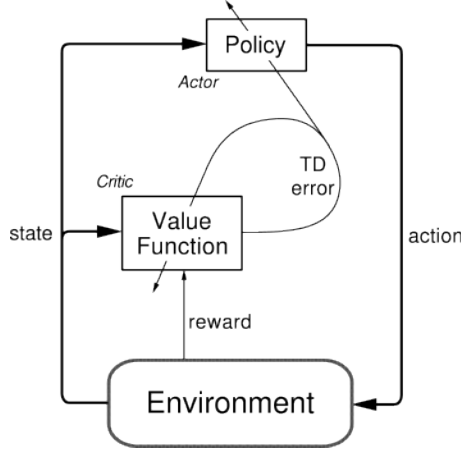


Figure 4.2: Visualization of the Actor-Critic structure

4.4 PROXIMAL POLICY OPTIMIZATION

Proximal Policy Optimization (PPO) is a popular algorithm in the field of reinforcement learning, introduced by OpenAI in 2017 as an improvement over previous policy optimization methods [20]. PPO takes its roots from the A2C algorithm and it has been designed to improve the stability and reliability of policy updates while maintaining simplicity of implementation. The algorithm addresses some of the issues related to traditional policy gradient methods, such as high variance and sensitivity to hyperparameters. It operates by updating the policy using a clipped objective function, which restricts how much the policy can change in a single update. This approach ensures that the new policy remains close to the old policy, promoting stable learning and avoiding drastic updates that could lead to performance degradation. The objective function for PPO can be expressed as:

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right], \quad (4.5)$$

where $r_t(\theta)$ is the probability ratio of the new policy to the old policy, $\hat{A}_t$ is the estimated advantage at time t , and ε is a small hyperparameter that controls the clipping range. The Advantage Function $\hat{A}_t$ measures how much better (or worse) an action taken in a particular state is compared to the expected return for that state. By quantifying the difference between the actual reward and the baseline (typically the value function), it helps guide the policy update to improve the policy over time. By using the minimum of the two expressions, PPO effectively limits the policy's update to a defined range, guaranteeing that it does not change too much

from the previous policy.

The algorithm for PPO can be summarized as follows:

Algorithm 4.2 Proximal Policy Optimization Algorithm

- 1: Initialize policy parameters θ and value function parameters w
- 2: **for** each iteration
- 3: Collect a set of trajectories by running the current policy $\pi(a|s; \theta)$
- 4: Compute advantages $\hat{A}_t$ using Generalized Advantage Estimation (GAE)
- 5: **for** each epoch
- 6: **for** each batch of data
- 7: Calculate the probability ratio:

$$r_t(\theta) = \frac{\pi(a_t|s_t; \theta)}{\pi(a_t|s_t; \theta_{old})}$$

- 8: Compute the clipped objective:

$$L^{CLIP}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right]$$

- 9: Update policy parameters by maximizing the objective:

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} L^{CLIP}(\theta)$$

- 10: Update value function parameters w by minimizing the loss:

$$w \leftarrow w - \alpha_v \nabla_w (V(s_t; w) - r_t)^2$$

11: **end for**

12: **end for**

13: **end for=0**

One of the significant advantages of PPO is its sample efficiency, allowing it to learn effectively from a limited number of samples. This efficiency is particularly useful in environments

where data collection is costly or time-consuming.

As shown in the algorithm, PPO is often implemented with additional techniques, such as advantage estimation and generalized advantage estimation (GAE), which further improve the stability and efficiency of the learning process. GAE, for example, provides a way to compute advantages over multiple time steps, balancing bias and variance in the estimates. Thanks to these improvements and variation of the "standard" algorithm, PPO has become one of the most popular techniques in deep reinforcement learning due to its robustness, ease of use, and solid performance among different tasks.

4.5 MONTE CARLO TREE SEARCH

Monte Carlo Tree Search (MCTS) is a powerful algorithm widely employed for decision-making in large and complex state spaces, particularly within the domain of game-playing AI. MCTS gained relevance thanks to its effectiveness in games like Go, where traditional approaches struggled to match human performance. The algorithm is really good at balancing the exploration of uncharted territories with the exploitation of known rewarding actions. Moreover, standard applications of MCTS typically do not incorporate neural networks, meaning that a dedicated training phase is unnecessary. While this characteristic may initially seem advantageous, it presents certain drawbacks. MCTS relies on simulations of the game to forecast potential outcomes, which can lead to significant time consumption when determining the best action at each move.

MCTS operates through four main phases: selection, expansion, simulation, and backpropagation. Each of these phases plays a crucial role in the algorithm's effectiveness. The Figure 4.3 represents a schematic overview of these steps and it's useful to better understand how the algorithm works.

SELECTION

The selection phase begins at the root node and traverses the tree down to a leaf node. During this traversal, the algorithm uses a tree policy, often based on the Upper Confidence bound for Trees (UCT) formula:

$$UCT = \frac{X}{N} + C\sqrt{\frac{\ln P}{N}}, \quad (4.6)$$

where:

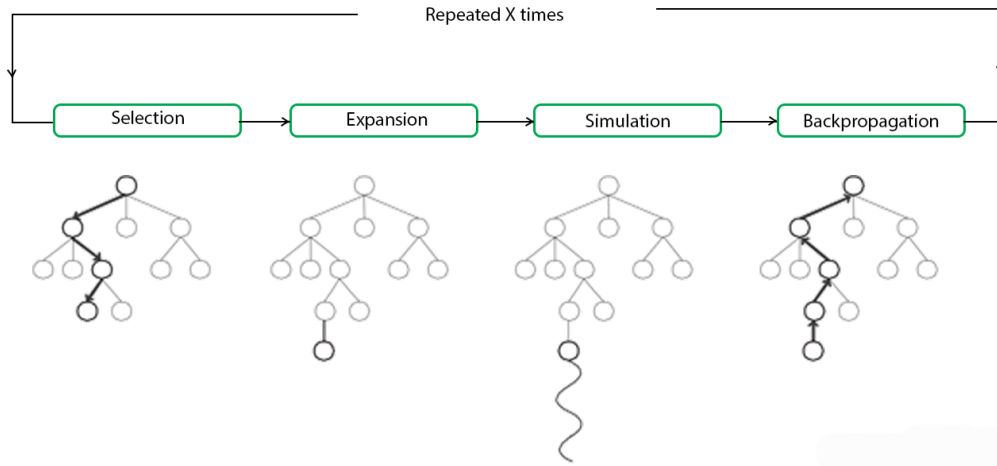


Figure 4.3: Visualization of MCTS steps

- X is the total reward of the node;
- N is the number of simulations for the node;
- P is the number of simulations for the parent node;
- C is a constant that controls exploration (usually equal to $\sqrt{2}$).

The selection process prefers nodes with high average rewards while also considering nodes that have been explored less frequently. This balance encourages exploration of potentially fruitful paths while still leveraging known information.

EXPANSION

Once a leaf node is reached, the expansion phase begins. This phase involves adding one or more child nodes to the tree, representing possible actions that can be taken from the current state. Not all leaf nodes are usually expanded; the decision to expand is often dependent on a predefined threshold or policy to manage computational resources effectively. Commonly, one action is chosen to expand based on the available legal moves, and new child nodes are created to represent these actions. Additionally, it is typical to implement strategies that prioritize expanding nodes that have not been explored frequently with the goal of promoting a more complete exploration of the search space.

SIMULATION

The simulation phase, often referred to as *rollout*, involves executing a random simulation from the newly expanded node until a terminal state is reached. This phase is crucial as it provides an estimate of the potential value of the state. The simulations can be performed using random playouts, where actions are chosen randomly, or by utilizing more sophisticated policies, depending on the desired accuracy and computational constraints.

BACKPROPAGATION

After a simulation concludes, the backpropagation phase updates the values of the nodes in the tree. The outcomes of the simulation (win, loss, or draw) are propagated back from the expanded node up to the root node, adjusting the value estimates for all nodes traversed during the selection phase. This update improves the accuracy of the value estimates for the respective actions, guiding future selections.

In conclusion, Monte Carlo Tree Search is a versatile and effective algorithm that has revolutionized the field of game-playing AI, indeed, thanks to its structure, popular algorithms like AlphaGo and AlphaZero have been developed. Its ability to balance exploration and exploitation, together with its scalability and adaptability, makes it a crucial tool for solving complex decision-making problems even almost 20 years later its first theorization [21].

4.6 ALPHAZERO

AlphaZero represents an innovative advancement in artificial intelligence, particularly in the domain of game-playing AI. Developed by DeepMind [11], AlphaZero builds upon its predecessor AlphaGo Zero and combines the principles of Monte Carlo Tree Search (MCTS) with deep neural networks in a new reinforcement learning framework. Unlike traditional game-playing agents, which require extensive human expertise or domain-specific knowledge, AlphaZero learns just through self-play, with minimal prior knowledge about the games it plays. The architecture of AlphaZero revolves around three core components: a neural network, Monte Carlo Tree Search (MCTS), and self-play. These elements work together to produce a highly efficient and generalized learning system, capable of mastering multiple games, such as Go, Chess, and Shogi, from scratch. Starting from the neural network, in AlphaZero

it is responsible for both policy and value estimation, similarly to A2C and PPO. It takes the current game state as input and outputs two key pieces of information:

- **Policy head:** A probability distribution over possible moves, guiding the search in MCTS.
- **Value head:** An estimate of the probability of winning from the current state.

The policy and value heads allow AlphaZero to make informed decisions during the tree search and to evaluate the quality of different states during self-play. The network is updated using data generated from MCTS-guided games, and this training process occurs iteratively as the agent improves. Thanks to the neural network the algorithm is able to expand fewer, but more relevant nodes. As result of this process we obtain a tree more expanded on depth and less on breath. As anticipated before, AlphaZero uses an enhanced version of Monte Carlo Tree Search, which is integrated with the neural network. During the selection phase, the policy head helps guide the search, focusing on the most promising actions rather than random exploration. The value head provides an accurate estimate of the quality of states, used during the backpropagation phase. It's also worth to underline that AlphaZero's learning is driven entirely by self-play. Starting from random moves, the agent plays games against itself, gradually improving through trial and error. The neural network is trained on data generated from these self-play games, and the resulting policy and value estimates become increasingly refined over time. As the network improves, the quality of MCTS-guided search also enhances, creating a feedback loop that leads to mastery of the game.

The algorithm of AlphaZero integrates these components in a loop of self-improvement. The main steps of the algorithm can be outlined as follows:

Algorithm 4.3 AlphaZero Algorithm

- 1: Initialize neural network f_θ , with random weights θ
 - 2: **for** each iteration
 - 3: **for** each game
 - 4: Play a game using MCTS guided by f_θ (policy head)
 - 5: Store the game data: states, actions, and outcomes
 - 6: **end for**
 - 7: Train f_θ using the collected data, minimizing the difference between the predicted value and the game outcome, and maximizing the probability of the chosen moves (policy head)
 - 8: **end for** =o
-

In AlphaZero, the decision of which nodes to expand during the Monte Carlo Tree Search (MCTS) is guided by a combination of exploration and exploitation strategies. The selection of child nodes from a parent node is based on the Upper Confidence Bound for Trees (UCT) formula (see Equation 4.6), which is the one used in MCTS, however it has been modified to include the values in output from the policy head. Indeed, the equation takes the name Predictor-Upper Confidence bounds for Trees (PUCT). This formula is pretty similar to the UCT as it is an extension of it. The PUCT formulation for selecting actions a from a state s is given by:

$$U(s, a) = Q(s, a) + C \cdot P(s, a) \cdot \frac{\sqrt{N(s)}}{1 + N(s, a)} \quad (4.7)$$

In this equation, $Q(s, a)$ represents the action-value estimate for taking action a in state s , while $P(s, a)$ denotes the prior probability of action a given state s , as predicted by the neural network. The term $N(s)$ indicates the total number of visits to the parent node s , and $N(s, a)$ is the number of visits to the child node resulting from action a . The constant C controls the degree of exploration. This approach allows AlphaZero to efficiently explore the game tree while guaranteeing that promising actions are prioritized for expansion, allowing the algorithm to improve its decision-making capacity over time.

Thanks to the brilliant work made by DeepMind, AlphaZero has represented a big jump in the development of general AI systems. By integrating Monte Carlo Tree Search with deep neural networks and relying on self-play, AlphaZero can achieve superhuman performance in multiple games without any domain-specific knowledge as the researchers have widely demonstrated.

5

Game Theory

5.1 INTRODUCTION TO GAME THEORY

Game Theory is a branch of mathematics that studies strategic decision-making in interactive situations. It provides a framework for analyzing scenarios where the outcome for each participant (or "player") depends not only on their own actions but also on the actions of others. Basically, game theory is the science of strategy, it offers tools to predict and analyze how rational individuals will behave in situations of conflict and cooperation.

The foundations of modern game theory came from the mid-20th century, although its roots can be traced back to earlier mathematical studies of probability and economics. A significant milestone took place in 1928 when John von Neumann published his influential paper on the theory of parlor games, which introduced the minimax theorem [22]. This was followed in 1944 by the collaboration of von Neumann and Oskar Morgenstern, who published *Theory of Games and Economic Behavior* [23], establishing game theory as a distinct field. Between 1950 and 1953, John Nash developed the concept of Nash Equilibrium, significantly expanding the applicability of game theory. The 1960s and 1970s saw further refinement of Nash's work by Reinhard Selten and John Harsanyi, who introduced concepts such as subgame perfect equilibrium and games with incomplete information. Together, these contributions have shaped game theory into a powerful analytical tool utilized across various disciplines.

In the context of this research, focusing on mastering Ultimate Tic Tac Toe through rein-

forcement learning, game theory provides valuable knowledge. It offers a framework for understanding the strategic depth of the game, the concept of optimal play, and the challenges of learning in a competitive, multi-agent environment. Game theory and Reinforcement Learning share a fundamental concern: how agents make decisions in interactive environments. Their relationship can be understood through several key concepts. Both fields address scenarios involving multiple decision-makers, referred to as players in game theory and agents in RL. Game theory provides a framework for understanding optimal strategies, which can inform the design of RL algorithms, particularly in multi-agent settings. In particular, the notion of Nash Equilibrium in game theory relates to the concept of stable policies in multi-agent RL, while Markov Games serve as a bridge between the two disciplines by extending Markov Decision Processes to make room for multi-agent scenarios. Additionally, concepts from game theory might also guide the design of reward structures in RL environments, promoting desired behaviors among agents.

5.2 BASIC CONCEPTS OF GAME THEORY

Game theory provides a framework for analyzing strategic interactions. To understand this framework, we need to familiarize with several key concepts: players, strategies, payoffs, and information sets. Each of these elements plays a crucial role in defining and analyzing games.

PLAYERS

Players are the decision-makers in a game. They can be individuals, groups, companies, or even automated agents, as in the case of reinforcement learning. In game theory, we typically assume players to be rational, meaning they act in their best interest to maximize their expected utility. Players can be categorized in various ways. Some games involve cooperative players who can form cooperative agreements, while others have non-cooperative players acting independently. Players might also be symmetric, having identical strategy sets and payoffs, or asymmetric, with different options or potential outcomes. In the context of Ultimate Tic Tac Toe, the players are typically two opponents competing against each other, each trying to get around the other to win the game.

STRATEGIES

A strategy is a complete plan of action a player will take given any situation that might happen in the game. Strategies can take different forms, depending on the nature of the game and the approach of the player. Pure strategies involve a player choosing a single action with certainty. In contrast, mixed strategies involve a player choosing a probability distribution over possible actions, introducing an element of randomness to their play. Some strategies may be dominant, providing the best outcome for a player regardless of what other players do. In Ultimate Tic Tac Toe, a pure strategy might be a complete plan specifying where to play in every possible game state. A mixed strategy, on the other hand, could involve randomizing between different moves in certain situations, making the player's actions less predictable.

PAYOFFS

Payoffs represent the outcome or utility a player receives from a particular combination of strategies chosen by all players. They are often represented in a payoff matrix, which shows the payoffs for each player for every possible combination of strategies. Games can be categorized based on their payoff structures. In zero-sum games, one player's gain is exactly balanced by the other player's loss. Non-zero-sum games, however, allow for different gains or losses, opening up possibilities for cooperation or more complex interactions. Payoffs are typically represented by utility functions, which are mathematical representations of players' preferences over different outcomes. These functions can be simple (like win/lose in a basic game) or more complex, considering multiple factors. For Ultimate Tic Tac Toe, the payoff could be binary (win/lose) or more complicated, considering factors such as the number of moves made or the number of sub-boards won. The choice of payoff structure can significantly impact the strategies employed by players or, in this case, the learning process of the reinforcement learning agent.

INFORMATION SETS

Information sets represent what a player knows when making a decision. This concept is crucial for distinguishing between different types of games and has significant implications for strategy and analysis. Games of perfect information are those where all players know the complete history of the game at every point. Chess is a classic example of this. In contrast, games of imperfect information are those where players may not know all previous actions or may not fully know the current game state. Poker, where players' cards are hidden from each other, is an example of imperfect information. Another important distinction is between games of complete

and incomplete information. In games of complete information, all players know the strategies and payoffs available to other players. Games of incomplete information involve uncertainty about other players' payoffs or available strategies. Ultimate Tic Tac Toe is a game of perfect information, as both players can see the entire board state at all times. This characteristic has important implications for the types of strategies that can be employed and the approaches that might be effective in training a RL agent to play the game.

5.3 TYPE OF GAMES

As explained in the previous section, games can be categorized in various ways based on their characteristics. Understanding these categories helps in analyzing game dynamics and developing appropriate strategies. Here, we'll explore three important classifications: zero-sum vs. non-zero-sum games, cooperative vs. non-cooperative games, and simultaneous vs. sequential games.

ZERO-SUM VS. NON-ZERO-SUM GAMES

One of the fundamental distinctions in game theory is between zero-sum and non-zero-sum games. This classification is based on the relationship between players' gains and losses. In zero-sum games, the combined payoff for all players is always zero, no matter which strategies they choose. This means one player's gain comes directly at the expense of the others. Chess and poker are classic examples of zero-sum games. In chess, for instance, a win for one player (+1) necessarily means a loss for the other (-1), and their sum is always zero. Non-zero-sum games, on the other hand, are those where the interplay of strategies can lead to net results different from zero. In these games, it's possible for all players to gain (a win-win situation) or for all to lose. Many real-world situations, such as trade negotiations, are non-zero-sum games. In these scenarios, cooperation can lead to mutual benefits, while non-cooperation might result in mutual losses. Ultimate Tic Tac Toe, like its simpler counterpart, is essentially a zero-sum game. A win for one player necessarily means a loss for the other.

COOPERATIVE VS. NON-COOPERATIVE GAMES

Another important classification is between cooperative and non-cooperative games. This distinction is based on the players' ability to form binding agreements. In cooperative games, players can form coalitions, make agreements, and coordinate their strategies. The focus in

these games is often on how gains from cooperation are distributed among the players. Examples include business partnerships or political alliances. Non-cooperative games, in contrast, assume that each player acts independently, without collaboration or communication with others. Most competitive scenarios, including many board games and sports, fall into this category. In these games, players cannot make agreements outside of those specifically modeled in the game. Ultimate Tic Tac Toe is played as a non-cooperative game. Each player makes their moves independently, trying to get around their opponent without any form of collaboration.

SIMULTANEOUS VS. SEQUENTIAL GAMES

Games can also be classified based on the timing of players' decisions, distinguishing between simultaneous and sequential games. In simultaneous games, players make their decisions at the same time, without knowledge of the other players' choices. The classic example is Rock-Paper-Scissors, where players reveal their choices simultaneously. These games are often represented using strategic form or normal form, typically visualized as a payoff matrix. Sequential games, also known as dynamic games, involve players making decisions in a specific order, with later players having some knowledge of earlier actions. Chess and Go are examples of sequential games. These are usually represented in extensive form, often visualized as a game tree. Ultimate Tic Tac Toe is a sequential game where players take turns making their moves. Each player can observe the opponent's previous move before deciding on their own. This sequential nature adds a strategic implications to the game, as players must consider not just the immediate consequences of their actions but also how they set up future possibilities or limit their opponent's options.

5.4 NASH EQUILIBRIUM

Nash Equilibrium is a fundamental concept in game theory, named after mathematician John Nash. It describes a state of a game where each player is making the best decision for themselves based on what their opponents are doing. Let's define a Nash Equilibrium as a set of strategies, one for each player, such that no player can unilaterally change their strategy to increase their payoff. In other words, given what the other players are doing, each player is playing their best response. Formally, for a game with n players, strategies $S = (S_1, \dots, S_n)$, and payoff functions $u_1, \dots, u_n$, a Nash Equilibrium is a strategy profile $s^* = (s_1^*, \dots, s_n^*)$ such that:

$$u_i(s_1^*, \dots, s_i^*, \dots, s_n^*) \geq u_i(s_1^*, \dots, s_i, \dots, s_n^*) \quad \forall s_i \in S_i, \quad \forall i = 1, \dots, n$$

This mathematical definition encapsulates the idea that no player can improve their outcome by unilaterally changing their strategy. In other words, no player has an incentive to deviate from their chosen strategy when the other player's strategy is maintained.

DOMINANT STRATEGIES AND BEST RESPONSES

To fully grasp the concept of Nash Equilibrium, it's important to understand two related ideas: dominant strategies and best responses. A dominant strategy is a strategy that provides the best outcome for a player regardless of what strategies other players choose. If a dominant strategy exists, it will always be part of a Nash Equilibrium. However, dominant strategies don't always exist in complex games. A best response is the strategy that produces the most favorable outcome for a player, given the strategies of the other players. In a Nash Equilibrium, each player's strategy is a best response to the other players' strategies.

FINDING NASH EQUILIBRIUM

Finding Nash Equilibrium can be straightforward in simple games, but it becomes increasingly complex in games with many players or strategies. Several common methods are employed to identify the Nash Equilibrium.

One approach is the *Iterative Elimination of Dominated Strategies*, where strategies that are consistently inferior to others are repeatedly removed, regardless of the actions of other players. Another method is *Best Response Dynamics*, which starts with an arbitrary strategy profile and iteratively updates each player's strategy to their best response until no player has an incentive to change their strategy. Additionally, *Support Enumeration* involves checking all possible combinations of pure strategies that could form the support of a mixed strategy Nash Equilibrium.

To illustrate the concept of Nash Equilibrium, let's consider the classic game theory example of the Prisoner's Dilemma. Two suspects are interrogated separately. Each can either confess (C) or remain silent (S). If both remain silent, they each serve 1 year. If both confess, they each serve 2 years. If one confesses and the other remains silent, the one who confessed goes free, while the silent one serves 3 years.

The payoff matrix (years in prison, lower is better) is:

	Suspect 2 (C)	Suspect 2 (S)
Suspect 1 (C)	$(-2, -2)$	$(0, -3)$
Suspect 1 (S)	$(-3, 0)$	$(-1, -1)$

Table 5.1: Payoff matrix for the Prisoner's Dilemma

In this case, confessing is a dominant strategy for both players. No matter what the other player does, each player is better off confessing. Therefore, (C, C) is the unique Nash Equilibrium of this game, even though both players would have a better payoff if they both remained silent.

In the context of Ultimate Tic Tac Toe, the concept of Nash Equilibrium represents a pair of strategies where neither player can improve their chances of winning by unilaterally changing their strategy. Due to the game's complexity, finding an exact Nash Equilibrium is computationally challenging. However, the concept of Nash Equilibrium is still valuable in this context. When training a reinforcement learning agent for Ultimate Tic Tac Toe, the goal is often to develop a strategy that approximates Nash Equilibrium play. This means creating an agent that plays in such a way that it's difficult for any opponent to consistently win against it.

6

Experimental Setup

This chapter presents the experimental framework developed to evaluate various reinforcement learning approaches in the context of Ultimate Tic Tac Toe. We begin by describing the computational infrastructure that supported our experiments, including both hardware specifications and software components. The sophisticated nature of training deep reinforcement learning agents necessitated substantial computational resources, which were provided through a shared high-performance computing environment.

Our experimental framework centers around a custom-built Ultimate Tic Tac Toe environment, designed to facilitate the training and evaluation of reinforcement learning agents. This environment implements the complete rule set of the game while providing the necessary interfaces for agent interaction, state observation, and reward calculation. The development of this environment required careful consideration of state representation, action space definition, and reward structure to enable effective learning.

In the following sections, we detail the custom Ultimate Tic Tac Toe environment. We show the hardware specifications of our computational setup, including the processing units, graphics cards, and memory configurations that facilitated our experiments. We then describe the software stack employed, highlighting the key frameworks and libraries that enabled the implementation of our reinforcement learning algorithms. This technical foundation was crucial in ensuring the reliability and reproducibility of our experimental results, while also providing the necessary computational efficiency for comprehensive agent evaluation.

6.1 THE ULTIMATE TIC TAC TOE ENVIRONMENT

The Ultimate Tic Tac Toe environment has been implemented as a Python class that provides a complete framework for reinforcement learning agents to interact with the game. The environment follows the standard OpenAI Gym interface conventions [24] while incorporating specific adaptations for the unique requirements of Ultimate Tic Tac Toe.

The environment's state space is represented by a 9×9 grid, implemented as a one-dimensional flatten NumPy array, where each cell can hold one of three values: 0 (empty), 1 (player one), or -1 (player two) as showed in Figure 6.1.

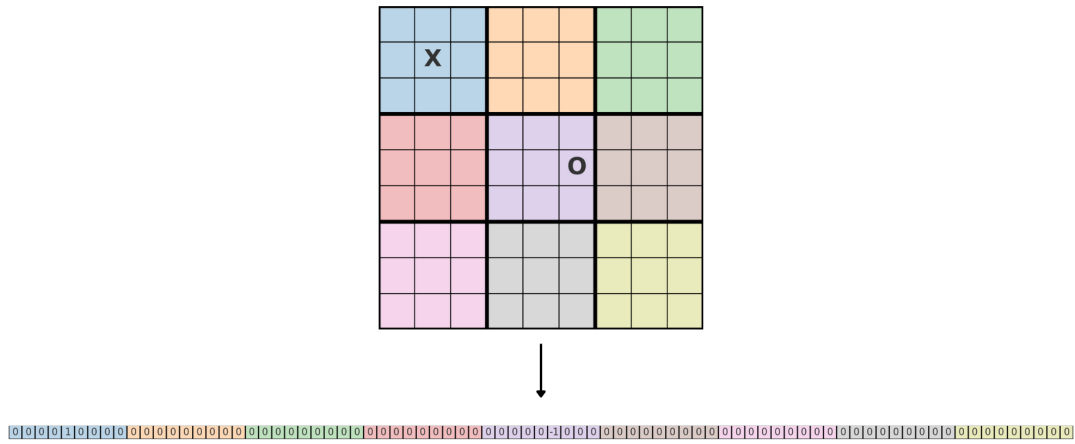


Figure 6.1: An example of a visual representation of the game state. The top section shows the board as seen by the player, while the bottom section illustrates the input format provided to the algorithms.

The action space consists of 81 possible moves, corresponding to each cell in the global grid, as showed in Figure 6.2. This representation allowed us to maintain a balance between computational efficiency and ease of interpretation for the learning agents.

The core game mechanics are managed through several key components. The state management system tracks both the local and global game states. It maintains the main 9×9 board configuration and a separate array for tracking sub-board victories. The environment also keeps track of the current player (alternating between 1 and -1) and stores the last move played, which is crucial for determining valid actions in subsequent turns. Furthermore, an action valida-

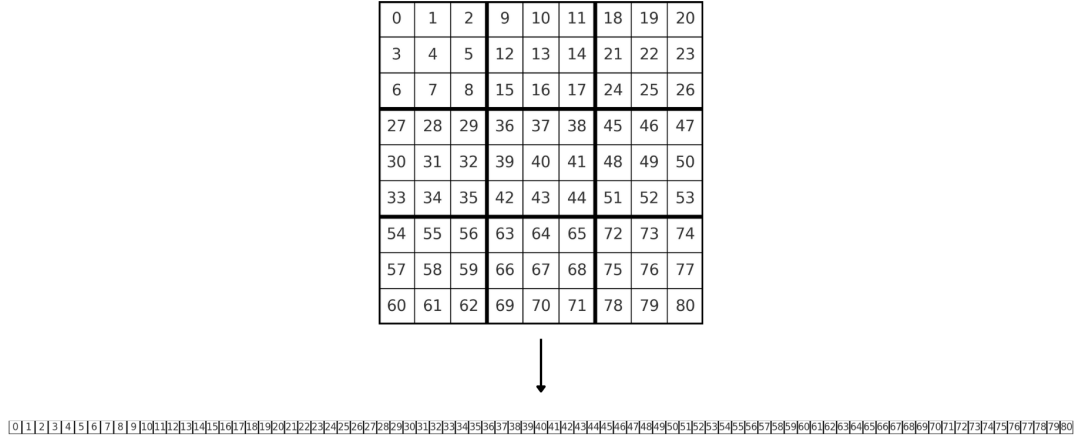


Figure 6.2: A visual representation of the game actions encoding.

tion system ensures game rule compliance through the `get_valid_actions` method. This method returns a binary array of length 81, where 1 indicates a legal move and 0 indicates an illegal move. The validation logic implements Ultimate Tic Tac Toe’s unique rules: players must play in the sub-board corresponding to the opponent’s last move’s position, unless that sub-board is won or full, in which case they may play in any available position on any unfinished sub-board.

The environment’s interaction model is built around the `step` method, which facilitates step-by-step progression in the game. This method performs several key functions: it validates and executes the agent’s chosen action, updates the game state accordingly, evaluates both local and global win conditions, manages the transition between player turns, and finally returns a tuple containing the updated state, the reward, a flag indicating whether the game has ended, and additional information.

The reward structure is minimalist and goal-oriented: agents receive a reward of 1 for winning the global game, -1 for losing it, and 0 otherwise. This sparse reward design encourages agents to develop strategies focused on achieving victory rather than intermediate objectives and also respect the zero-sum game concept represented by Ultimate Tic Tac Toe.

The environment includes several utility features that facilitate training and evaluation:

- A `reset` method that reinitializes the game state while alternating the starting player

- A `render` method that provides a human-readable ASCII representation of the current board state
- A `clone` method that creates deep copies of the environment, enabling tree search algorithms and self-play training
- A `to_state` method that returns a flattened representation of the board state and valid actions, suitable for neural network processing

The win-checking mechanism operates at two levels: local (`check_win`) and global (`check_global_win`). Both methods examine horizontal, vertical, and diagonal win conditions, with the global checker operating on the sub-board victory array rather than individual cell values. This hierarchical structure efficiently manages the game’s nested nature while maintaining clear victory condition evaluation.

This implementation provides a robust foundation for training reinforcement learning agents, offering both the necessary game mechanics and the flexibility to accommodate various learning approaches. The environment’s design prioritizes computational efficiency while maintaining the complete rule set of Ultimate Tic Tac Toe, making it suitable for extensive training sessions and experimental evaluations.

6.2 ENVIRONMENT SETUP

The experimental work in this thesis was conducted using a comprehensive computational infrastructure provided by the University of Padua, consisting of both hardware and software components that enabled efficient training and evaluation of our models. The experiments were performed on a shared computational server with the following specifications:

- **CPU:** AMD Ryzen Threadripper PRO 5995WX, 64 cores, 128 threads and maximum clock speed of 7.02 GHz
- **GPU:** Three NVIDIA RTX A6000 units, 48 GB VRAM per unit
- **Memory:** 503 GB RAM

All the software produced has been written in Python and we exploited different libraries to train the models and manage the data. The implementation relied on the following software components:

- **Python** (v3.11.9)

- **TensorFlow** (v2.12.0)
- **Keras** (v2.12.0)
- **NumPy** (v1.23.5)

This robust computational infrastructure provided an ideal environment for conducting comprehensive reinforcement learning experiments, allowing for systematic evaluation of various algorithms and hyperparameter configurations while maintaining reasonable training times. Concluding, all the code produced and used for this experiment has been made publicly available for reference and for future works [25].

7

Results

The preceding chapters established the theoretical foundations of reinforcement learning, described the game environment of Ultimate Tic Tac Toe, and explored relevant game theory concepts. This chapter bridges theory and practice by presenting an empirical investigation into the performance of various reinforcement learning agents in the Ultimate Tic Tac Toe domain. Through a series of carefully designed experiments, we evaluate the effectiveness of multiple state-of-the-art reinforcement learning algorithms, including Deep Q-Networks (DQN), Double Deep Q-Networks (DDQN), Actor Critic (A2C), and Proximal Policy Optimization (PPO), in mastering this challenging strategic game. The primary objectives of our experimental investigation are triple. First, we aim to train multiple reinforcement learning agents via self-play in the game of Ultimate Tic-Tac-Toe, using a unified environment setup to ensure comparability. Second, we seek to identify the most effective agents through a structured evaluation process based on head-to-head tournaments and Elo rating dynamics, allowing us to rank their relative performance objectively. Third, we focus on refining and incrementally improving the top performing architectures through targeted modifications, such as data augmentation or architectural depth, with the goal of converging on a robust and strategically competent final agent.

7.1 MODELS SETUP

The training methodology for these models relies on self-play, a technique where the model learns by playing against itself. This approach maximizes the efficiency of data utilization from the environment, as every game played generates valuable training examples. A critical aspect of the training process is how we handle game states, particularly in games like Ultimate Tic Tac Toe where players take turns. The state management becomes complex because when Player 1 makes a move, they transition the game from State A to State B. However, from Player 1's perspective, the next meaningful state they'll encounter (State C) won't occur until after Player 2 has made their move. This creates a delay in the learning process, as Player 1's model can't immediately update its understanding of how good its move was until it sees the resulting position after Player 2's response. The sequence flows from State A, where it's Player 1's turn, through their move to State B, where Player 2 then makes their move, finally arriving at State C where Player 1 can act again. To properly train the model, we need to account for this delay in feedback. The model needs to learn not just from the immediate result of its moves, but from the subsequent state that comes out after the opponent's response.

In our experimental setup, while various hyperparameters were model-specific, we maintained consistent environmental conditions and evaluation protocols across all models. Each model was trained for 600,000 iterations in an identical environment. During training, we used batches of 256 simulated games in parallel to accelerate learning and improve sample efficiency. Performance evaluation was performed at regular intervals of 1,000 iterations, with each model tested against a baseline random agent to ensure a standardized comparison.

7.1.1 DQN RESULTS

The Deep Q-Network approach utilize a single neural network to learn the so-called Q-table, as already explained in Chapter 2. We tested the ability of this model to learn to play Ultimate Tic Tac Toe. This are the training details implemented:

- Neural network: 2 layers of 256 neurons each, ReLu as activation function
- Learning rate: 0.0003
- Decaying exploration rate regulated by the following formula:

$$\text{new_exploration_rate} = 0.05 + (1 - 0.05) \cdot e^{-0.0001 \cdot \text{episode}} \quad (7.1)$$

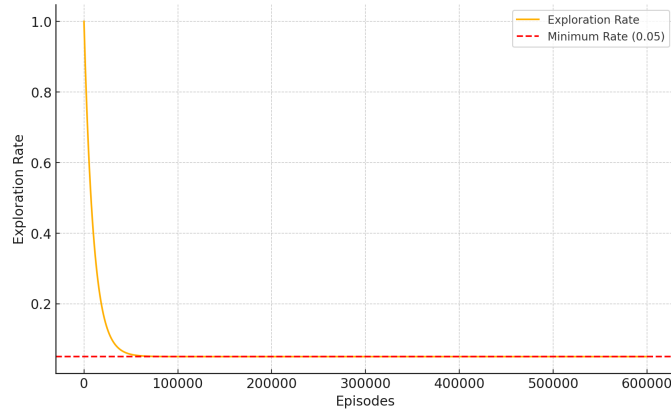


Figure 7.1: Plot of the exploration rate.

The results in Figure 7.2 indicate that the DQN model performs exceptionally well against a random agent. Initially, the win rate starts relatively low but improves rapidly during the early stages of training, surpassing a win rate of 70% within the first 100,000 iterations. Beyond 200,000 iterations, the model demonstrates consistent performance, achieving a win rate exceeding 90%, with a minor loss rate and a stable draw rate.

This progression reflects the model's ability to quickly learn strategies while maintaining them over subsequent training iterations. The consistent dominance of the green bars (win rate) over the red bars (loss rate) further confirms that the agent effectively exploits the random agent's weaknesses, reinforcing the suitability of the DQN approach for Ultimate Tic Tac Toe.

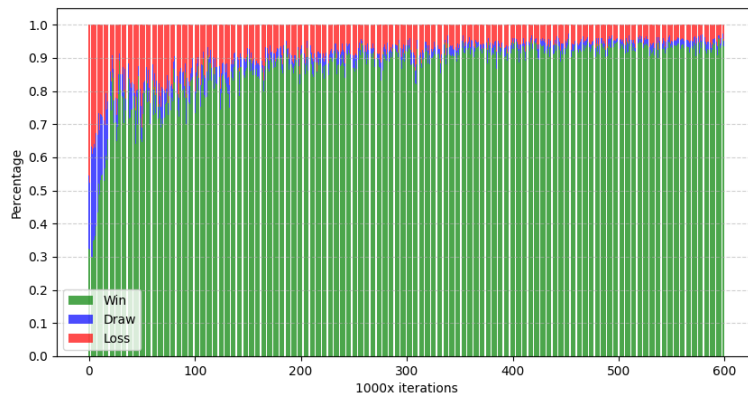


Figure 7.2: This chart represent the performances of the DQN model assessed every 1000 iterations against a random agent on 1000 simulated games. In green, we represent the win rate of the trained agent. In blue, we represent the draw rate. In red we represent the loss rate.

7.1.2 DDQN RESULTS

The Double Deep Q-Network (DDQN) approach addresses the overestimation bias observed in traditional DQN by decoupling action selection from action evaluation, as discussed in Chapter 2. This improvement improves the stability and performance of Q-learning in complex environments. We tested the ability of this model to learn to play Ultimate Tic Tac Toe. These are the training details implemented:

- Neural networks: 2 layers of 256 neurons each, both for the main model and the target one
- ReLu as activation function
- Batch normalization (before linear activation)
- Learning rate: 0.0003
- Decaying exploration rate regulated by Formula 7.1, which is the same used in the DQN approach

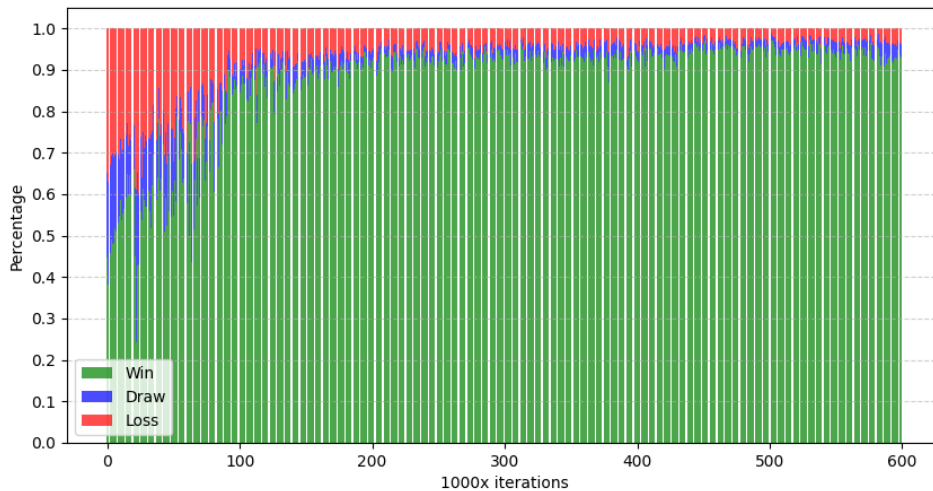


Figure 7.3: This chart represent the performances of the DDQN model assessed every 1000 iterations against a random agent on 1000 simulated games. In green, we represent the win rate of the trained agent. In blue, we represent the draw rate. In red we represent the loss rate.

The results presented in Figure 7.3 illustrate the effectiveness of the Double Deep Q-Network (DDQN) approach. The performance trajectory is similar to the DQN model but shows improved stability and convergence. The win rate starts low but rapidly increases within the first 100,000 iterations, eventually exceeding 90%. Unlike the DQN results, the DDQN model demonstrates slightly better control over overestimation, as evidenced by the more gradual reduction in the loss rate (red bars) and the consistent draw rate (blue bars). Beyond 200,000 iterations, the model achieves a dominant win rate, with losses becoming nearly negligible.

7.1.3 A2C RESULTS

The Advantage Actor-Critic (A2C) approach leverages a synchronous advantage estimation mechanism to stabilize the training of policy and value networks. Unlike Q-learning-based methods, such as DQN and DDQN, A2C is a policy gradient algorithm that directly optimizes the policy while minimizing the variance in learning through the critic network. This approach usually enables more efficient learning in environments with continuous or large discrete action spaces.

We implemented the A2C algorithm for Ultimate Tic Tac Toe using separate networks for the actor (policy) and the critic (value function). The details of the implementation are as follows:

- Actor and Critic networks: Both use two layers of 256 neurons each, with ReLU as the activation function.
- Batch normalization is applied before the activation functions in each layer to improve convergence.
- The actor network outputs a probability distribution over available actions using a softmax activation function.
- Learning rates: 0.0003 for the actor network and 0.0001 for the critic network.
- Entropy regularization with a weight of 0.01 to encourage exploration.
- A discount factor of 0.99 is used for temporal difference updates.

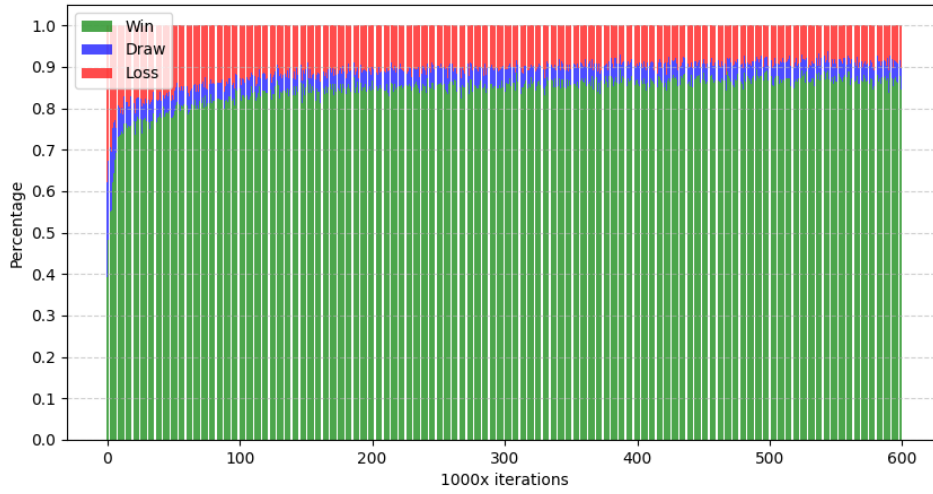


Figure 7.4: This chart represents the performances of the A2C model assessed every 1000 iterations against a random agent on 1000 simulated games. In green, we represent the win rate of the trained agent. In blue, we represent the draw rate. In red, we represent the loss rate.

Figure 7.4 illustrates the progression of the A2C agent’s performance. The win rate (green) starts at a modest level and grows steadily, eventually stabilizing above 80%. Meanwhile, the draw rate (blue) exhibits a slight increase but remains consistent, while the loss rate (red) drops significantly, demonstrating the agent’s improving ability to outperform the random opponent. Unlike the DQN and DDQN models, the A2C agent achieves rapid convergence with a more balanced final outcome. This highlights the stability and effectiveness of the A2C algorithm for optimizing both the policy and the value function in a synchronous way.

7.1.4 PPO RESULTS

The PPO agent was implemented using a clipped surrogate objective to balance policy updates with stability, an entropy term to promote exploration, and separate actor-critic networks to optimize policy and value estimates independently. The architecture and hyperparameters are detailed below:

- The actor network consists of two fully connected layers with 256 hidden units each, ReLU activations, and a softmax output layer to produce the action probabilities.
- The critic network has the same architecture as the actor but outputs a single scalar value representing the estimated value of a state.

- The learning rate for the actor network is 1×10^{-5} , while for the critic network, it is 1×10^{-4} .
- The PPO clipping factor ϵ is set to 0.2, ensuring stable and restrained policy updates.
- We tried two entropy regularization terms with a weight of 0.01 and another training with a weight of 0.00 to verify how the model would respond to the change.
- The discount factor γ is 0.99, promoting long-term reward optimization.
- The GAE lambda λ is 0.95, enabling smoother and more stable advantage estimation.

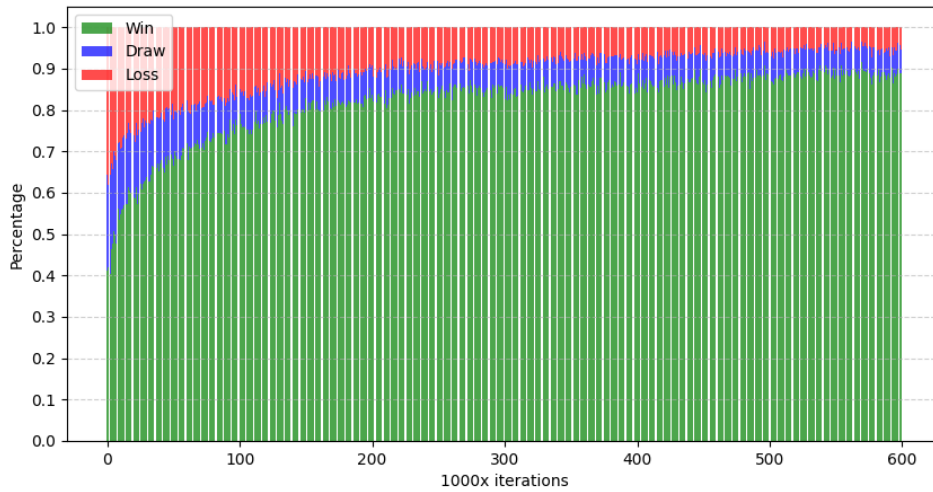


Figure 7.5: Performance of the PPO agent assessed every 1000 iterations against a random agent on 1000 simulated games. The green bars represent the win rate, blue bars represent the draw rate, and red bars represent the loss rate.

Figure 7.5 demonstrates the performance of the PPO agent over time. The win rate (green) rapidly increases in the early iterations, surpassing 90% after approximately 500k iterations for the first time. The draw rate (blue) stabilizes near 5%, while the loss rate (red) remains below 5% in the final stages, indicating that the agent successfully minimizes losses against a random opponent. Compared to A2C, the PPO agent achieves slightly faster convergence and better overall performance, attributable to the stability and robustness of the clipped PPO objective. This result highlights the superiority of PPO in scenarios requiring fine-grained policy optimization and stable training dynamics.

7.2 BEST MODEL RESEARCH

Having established that all four algorithms (DQN, DDQN, A2C, and PPO) can learn to convincingly beat a naive random opponent, we next turn to identifying which agent truly emerges as the strongest when they face off against one another or an evenly matched baseline. In this section, we present two complementary evaluations:

1. **Free-for-All Tournament:** A round-robin “grand tournament” in which each trained agent plays every other trained agent for 2,000 games. To ensure fairness, each agent starts as the first player in exactly 50% of the games. In addition to comparing self-play-trained agents, we include two search-based agents in the tournament: AlphaZero exploiting the PPO trained network and Monte Carlo Tree Search (MCTS). This allows us to directly measure head-to-head strength across a diverse range of approaches, without relying solely on indirect metrics like win rates against random agents.
2. **Elo Rating System:** An adaptive, statistically principled ranking of agents based on pairwise match results. By assigning and iteratively updating Elo scores, we produce a single scalar “skill” rating for each agent that reflects win/loss/draw performance against the other agents.

7.2.1 FREE-FOR-ALL TOURNAMENT

Each ordered pair of distinct agents produced three empirical frequencies: win rate, draw rate, and loss rate for the row agent. Figure 7.6 displays these compositions in green (win), gray (draw), and red (loss). Figure 7.7 then translates them into an expected-score advantage matrix, defined as $(\text{win}\% - \text{loss}\%)$.

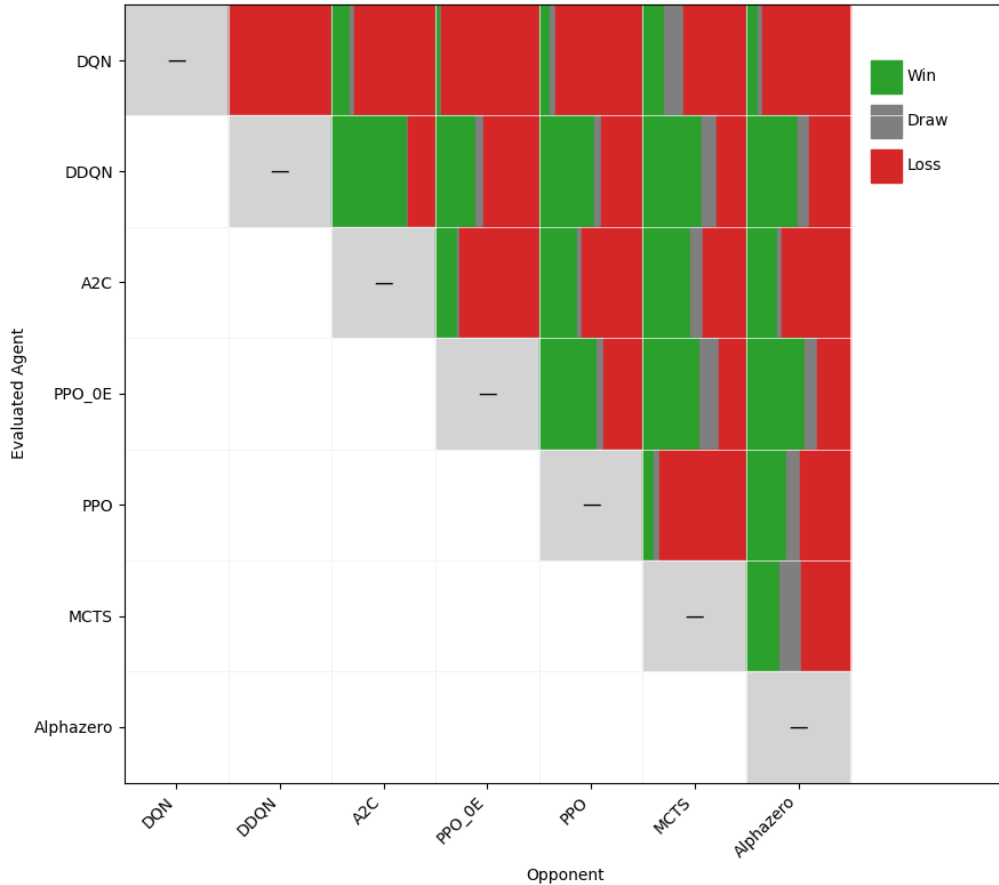


Figure 7.6: Empirical outcome frequencies for each agent pairing, where rows represent the evaluated (“row”) agent and columns represent the opponent. Each cell visualizes the proportion of results—green for wins, gray for draws, and red for losses—computed 2000 games per matchup.

Inspection of these figures reveals that PPO_0E, which represent the PPO agent trained with zero entropy, convincingly outperforms standard PPO, achieving a 55.4% win rate against it compared to PPO’s 37.0%. DDQN ranks second overall, defeating both MCTS and Alphazero in direct play, though it falls short when paired against either PPO variant. MCTS and Alphazero occupy the middle ground, trading wins and losses with similar frequency. The A2C agent

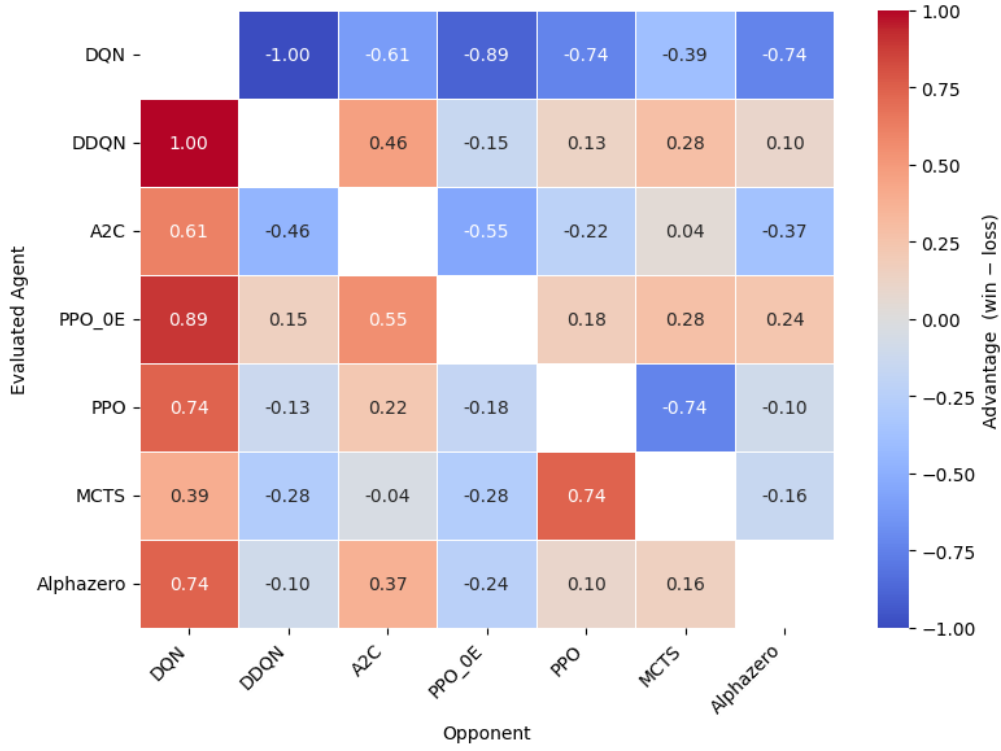


Figure 7.7: Expected-score advantage matrix: each cell shows (win%-loss%) for the row agent versus the column agent, color-coded from red (strong advantage) through white (neutral) to blue (strong disadvantage).

manages occasional upsets but generally loses more than half its games, while vanilla DQN fails to win even once against DDQN and otherwise posts the weakest results.

7.2.2 ELO RATING

To summarize the pairwise tournament results into a single scalar ranking, we implemented a batch Elo update procedure in Python. All seven agents (DQN, DDQN, A2C, PPO, PPO_OE, MCTS, and Alphazero) were initialized at 1500 Elo points and updated iteratively based on their aggregated head-to-head outcomes. For each matchup, we used the win/draw/loss proportions over 1,000 games to compute Elo adjustments in batch mode, rather than after individual games. Each agent’s Elo was updated according to the standard formula:

$$\text{NewRating}_A = \text{OldRating}_A + K \cdot (\text{Score}_A - \text{Expected}_A)$$

where Score_A is the empirical score of agent A (with wins = 1, draws = 0.5), and $\text{Expected}_A = \frac{1}{1+10^{(R_B-R_A)/400}}$ is the expected score based on the opponent's rating R_B . Since all ratings influence each other, we applied updates across the full set of 42 matchups (replicated in both directions) and repeated the process over 50 passes. The K-factor, set to 20, controls the magnitude of adjustments and helps ratings stabilize. This iterative, global update procedure ensures the final ratings reflect the transitive relationships between agents, mitigating order bias and cyclic dominance effects. The final Elo standings, rounded to one decimal place, appear in Table 7.1 and are visualized in Figure 7.8.

Agent	Final Elo
PPO_0E	1633.2
DDQN	1604.7
Alphazero	1565.5
MCTS	1532.5
PPO	1493.3
A2C	1452.6
DQN	1218.2

Table 7.1: Final Elo ratings after iterative batch updates (K=20, 50 passes).

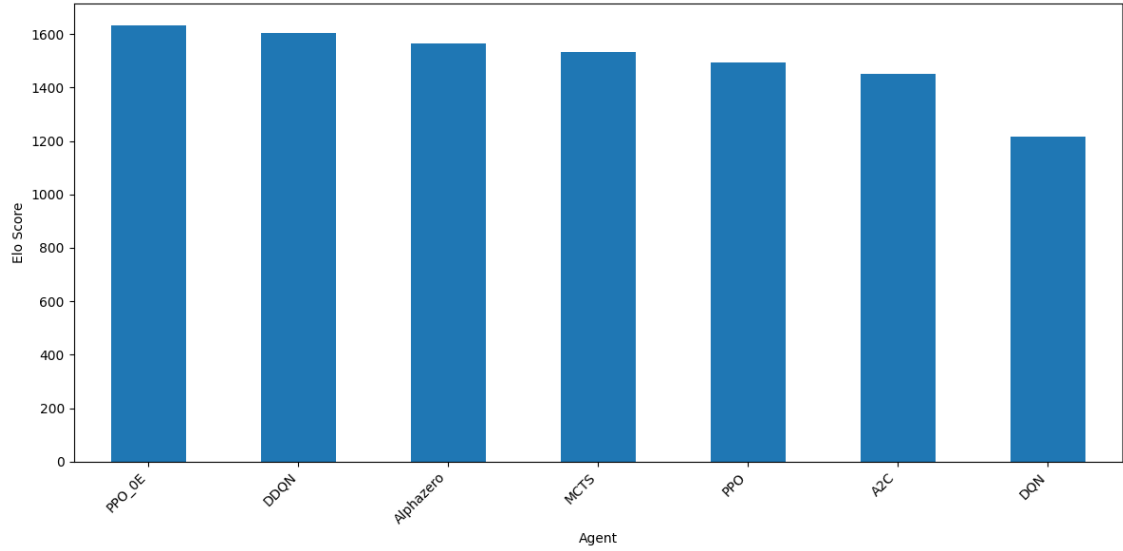


Figure 7.8: Bar chart of Elo scores by agent, sorted from highest (PPO_0E) to lowest (DQN).

These results confirm the hierarchy observed in the free-for-all tournament:

PPO with zero entropy (PPO_0E) > DDQN > Alphazero > MCTS > PPO > A2C > DQN.

7.2.3 ADDITIONAL IMPROVEMENTS

The free-for-all experiments in Section 7.2 identified DDQN and PPO_0E as the strongest baseline agents. To explore the margin still available, two additional possible improvements were investigated: a deeper residual network backbone and rotational data augmentation.

RESIDUAL NETWORK

Both DDQN and PPO have been re-implemented with a six-block dense Residual Network (ResNet) of width 256, as shown in Figure 7.9.

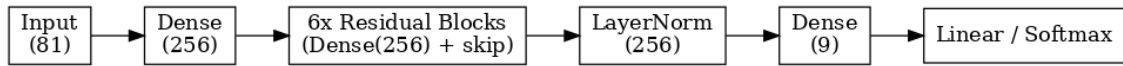


Figure 7.9: Visual representation of the residual network exploited in our experiment.

Layer normalization was applied after the final residual block, followed by the task-specific output layer (linear for DDQN, softmax for PPO). The deeper architecture enhances the model's representational capacity, while the residual skip connections help maintain stable gradients during training. It is important to note that all networks used in both approaches adopted this residual design. In addition, the learning rates were reduced since we observed instability and overshooting when using the original learning rates from Section 7.1. The parameters used for the two approaches are as follows:

- **DDQN:** exponentially decaying exploration rate, learning rate set to $3e - 5$, and soft target network updates with a target update frequency of 20.
- **PPO:** actor learning rate set to $1e - 6$, critic learning rate set to $1e - 5$, clip epsilon of 0.2, discount factor γ equal to 0.99, GAE parameter λ equal to 0.95, and entropy weight set to zero.

Similarly to what we showed in Section 7.1, from Figure 7.10 and 7.11 you can respectively observe the learning curves of the new models named: DDQN_resnet and PPO_0E_resnet.

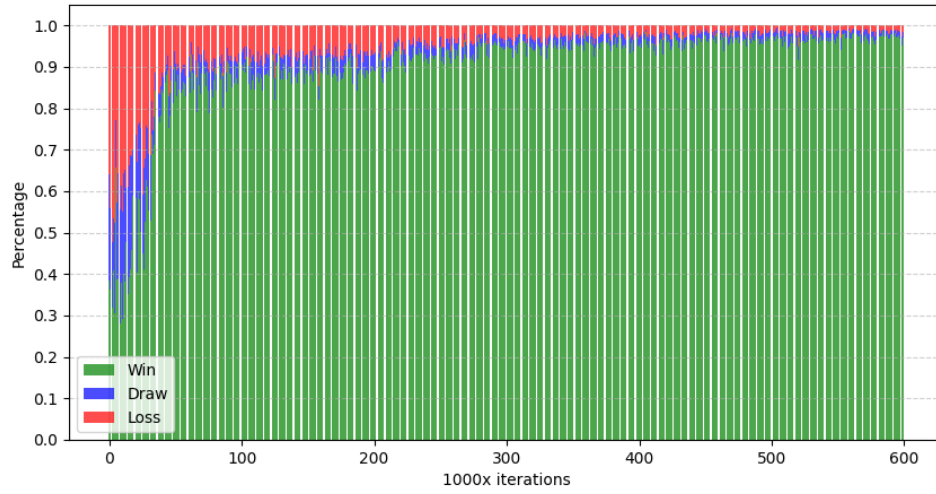


Figure 7.10: This chart represents the performances of the DDQN_resnet model assessed every 1000 iterations against a random agent on 1000 simulated games. In green, we represent the win rate of the trained agent. In blue, we represent the draw rate. In red, we represent the loss rate.

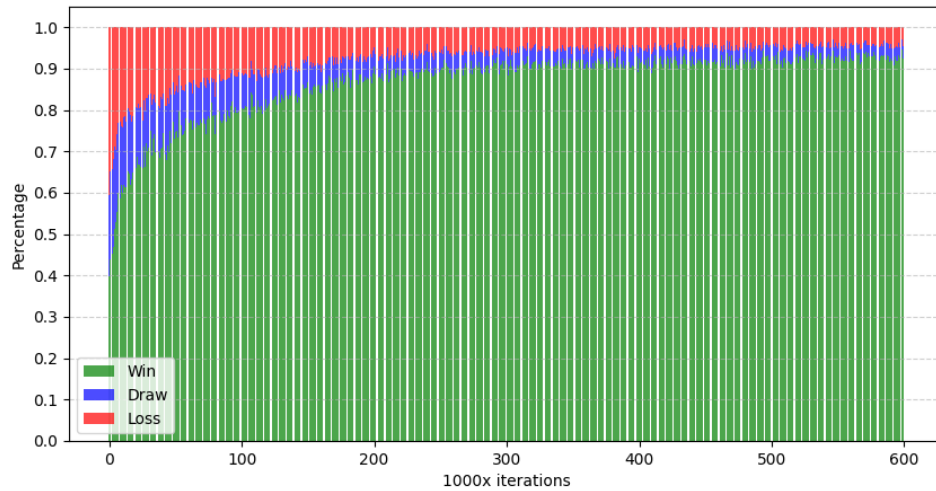


Figure 7.11: This chart represents the performances of the PPO_0E_resnet model assessed every 1000 iterations against a random agent on 1000 simulated games. In green, we represent the win rate of the trained agent. In blue, we represent the draw rate. In red, we represent the loss rate.

A comparison of Figures 7.10 and 7.11 shows that the six-block ResNet backbone unlocks clear gains for the value-based agent: DDQN_resnet surpasses a 90 % win-rate after roughly half

a million iterations, while its loss-rate is cut by more than half relative to the vanilla network. The policy-gradient counterpart, PPO_0E_resnet, also benefits, achieving a steadier ascent and a lower loss-rate, but its curve reaches a plateau earlier, suggesting a capacity vs over-fitting trade-off that we revisit in Section 7.4. Overall, these results confirm that deeper residual backbones provide additional performance headroom, particularly for DDQN, and motivate the next step of exploiting rotational augmentation to push the efficacy further.

DATA AUGMENTATION

Ultimate Tic-Tac-Toe has rotational symmetry, which can be used to improve generalization. During training, each sampled state–action pair was augmented by applying the four possible board rotations: 0° , 90° , 180° , and 270° , as shown in Figure 7.12.

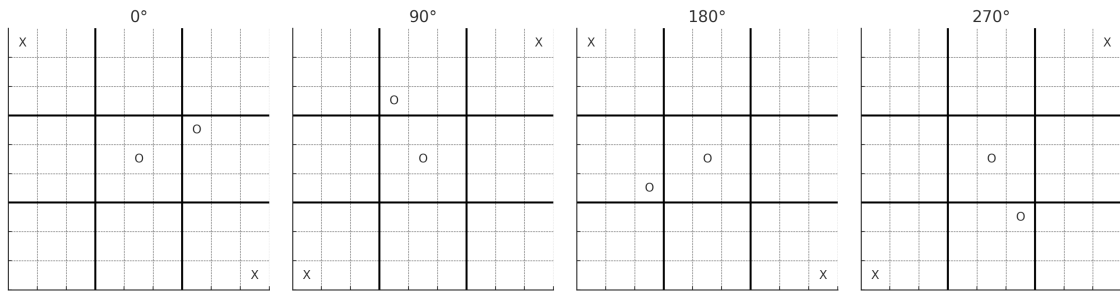


Figure 7.12: Visual representation of rotational data augmentation. Starting from the original board configuration (0°), we generate three additional training samples by rotating the board by 90° , 180° , and 270° .

As a result, each original sample generated four logically equivalent versions, allowing the network to “see” and learn from four different states by playing one single game. It’s important to mention that we add the data augmentation in the training process after the residual network implementation, so we simply used the same parameters as previously explained in this Section. From Figure 7.13 and 7.14 you can respectively observe the learning curves of the new models named: DDQN_resnet_data_aug and PPO_0E_resnet_data_aug.

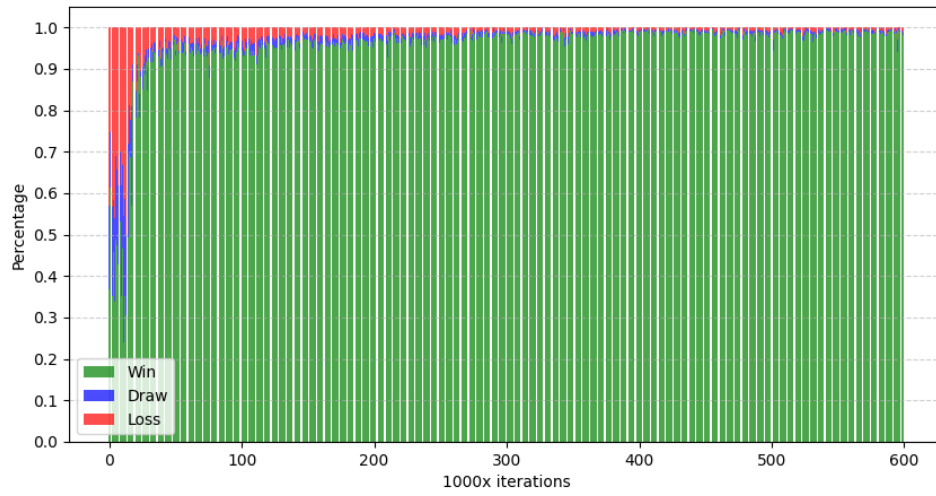


Figure 7.13: This chart represents the performances of the DDQN_resnet_data_aug model assessed every 1000 iterations against a random agent on 1000 simulated games. In green, we represent the win rate of the trained agent. In blue, we represent the draw rate. In red, we represent the loss rate.

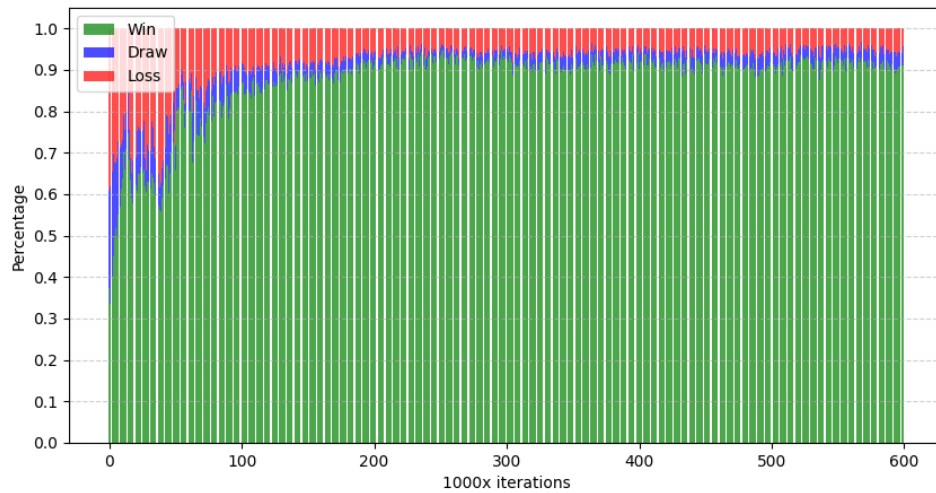


Figure 7.14: This chart represents the performances of the PP0_0E_resnet_data_aug model assessed every 1000 iterations against a random agent on 1000 simulated games. In green, we represent the win rate of the trained agent. In blue, we represent the draw rate. In red, we represent the loss rate.

ELO RE-RANKING

The extended free-for-all tournament was repeated with the new variants included. Figure 7.15 visualizes the updated win/draw/loss composition and the expected score matrix; the qualitative hierarchy is unchanged but a clear new leader emerges.

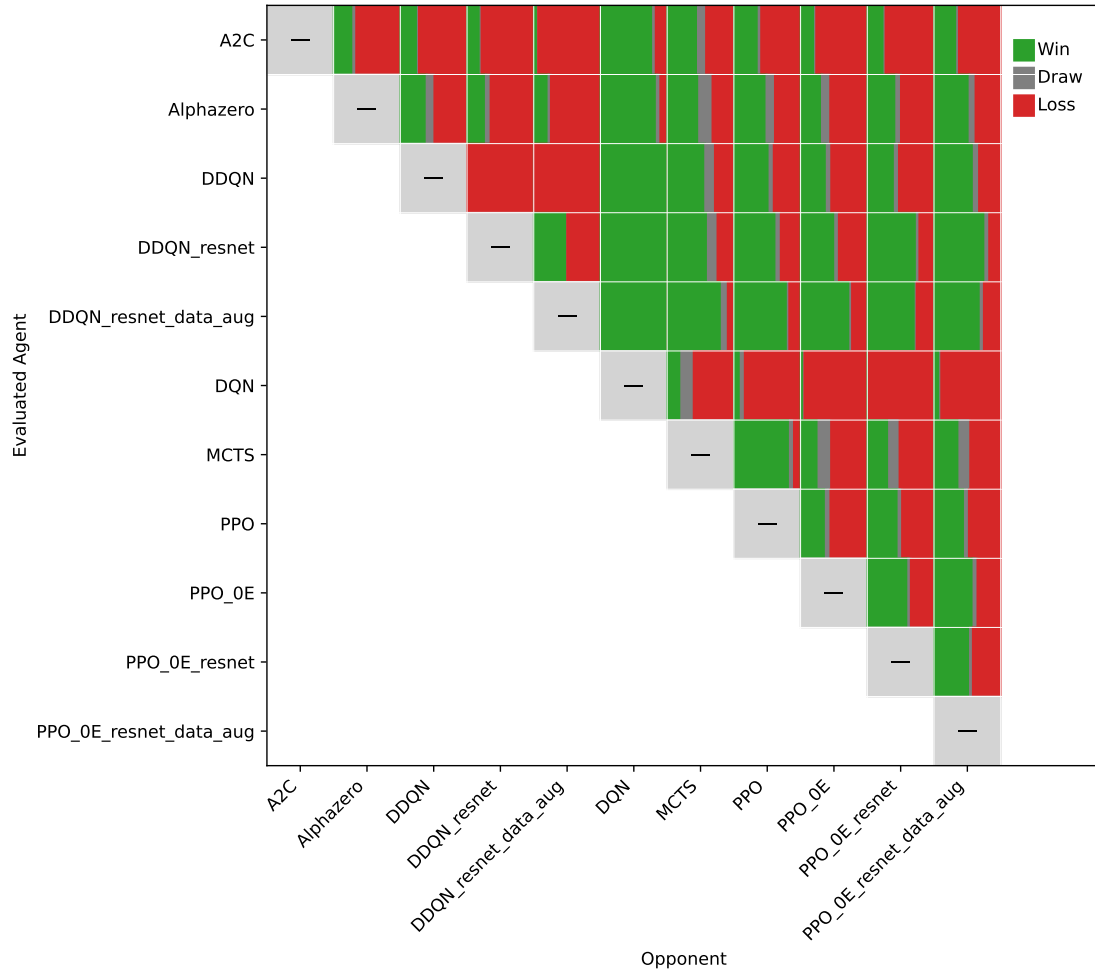


Figure 7.15: Empirical outcome frequencies for each agent pairing, where rows represent the evaluated (“row”) agent and columns represent the opponent. Each cell visualizes the proportion of results—green for wins, gray for draws, and red for losses—computed 2000 games per matchup. In this chart we added DDQN_resnet, DDQN_resnet_data_aug, PPO_0E_resnet, PPO_0E_resnet_data_aug.

In addition to Figure 7.15, Table 7.16 and Figure 7.17 present the new Elo scores.

Agent	Elo
DDQN_resnet_data_aug	1750.4
DDQN_resnet	1692.9
PPO_0E	1585.1
PPO_0E_resnet	1529.9
Alphazero	1517.6
DDQN	1497.9
PPO_0E_resnet_data_aug	1485.0
MCTS	1473.2
PPO	1464.6
A2C	1385.6
DQN	1117.9

Figure 7.16: Final Elo ratings for all agents.

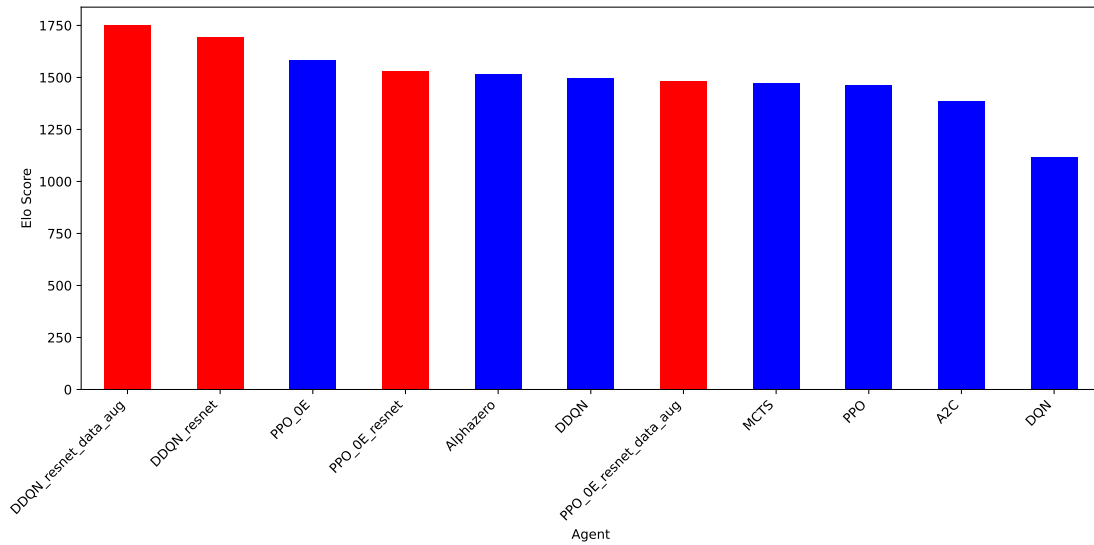


Figure 7.17: Bar chart of Elo scores by agent, sorted from highest to lowest. In red there are the new trained models: DDQN_resnet, DDQN_resnet_data_aug, PPO_0E_resnet and PPO_0E_resnet_data_aug.

From a first look at the data, we can state that the DDQN agent equipped with both the residual backbone and rotational augmentation surpasses the 1700-point milestone, a gain of more than 150 Elo over the previous champion. Interestingly, PPO appears to overfit to the deeper architecture, losing ten Elo points when augmentation is applied. DDQN_resnet_data_aug

not only reaches the top position from policy-gradient methods but does so with a 50-point margin over the next best competitor, demonstrating that carefully chosen architectural and data-centric improvements can considerably improve the performance and let the agent explore new game’s logic.

FINAL AGENT TRAINING AND EVALUATION

Given that DDQN_resnet_data_aug achieved the highest Elo rating among all evaluated models, we conducted a final training run to develop the strongest possible agent using this architecture and training pipeline. This extended training was performed over 2 million iterations to allow the agent to explore a larger and more diverse set of game situations.

Considering that each match can last up to 81 moves, and that training was conducted with 256 games simulated in parallel—further augmented by four rotational transformations per board, the final agent effectively experienced over 25 millions full games.

Figure 7.18 presents the performance of the resulting agent, referred to as DDQN_resnet_data_aug_2m1n, against a random opponent across the training process. The evaluation was conducted every 1,000 iterations using 1,000 simulated games per checkpoint.

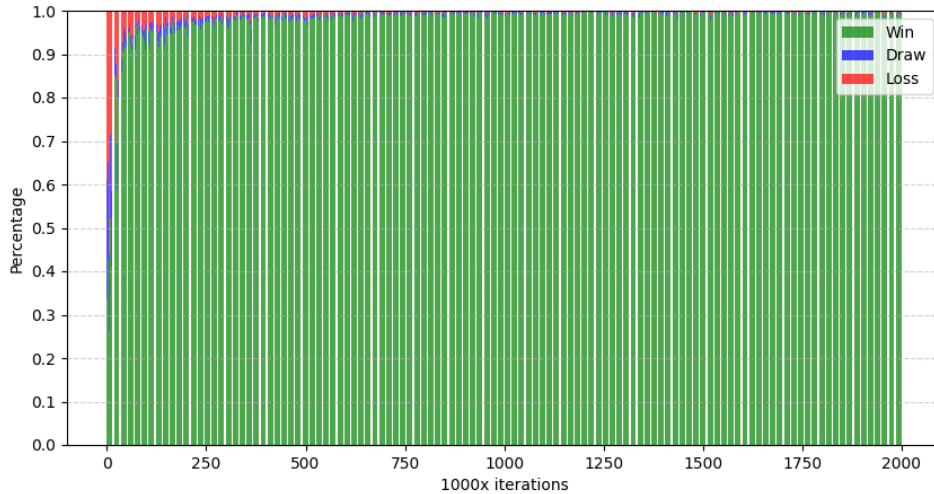


Figure 7.18: Performance of the DDQN_resnet_data_aug_2m1n model evaluated every 1000 iterations against a random agent over 1000 games. The green bars represent the win rate, blue bars the draw rate, and red bars the loss rate.

To assess its strength in a competitive setting, we included this final agent in a new final free-for-all round-robin tournament featuring the previously trained agents. The outcome fre-

quencies from these matchups are shown in Figure 7.19, while the updated Elo rankings are presented in Figure 7.20.

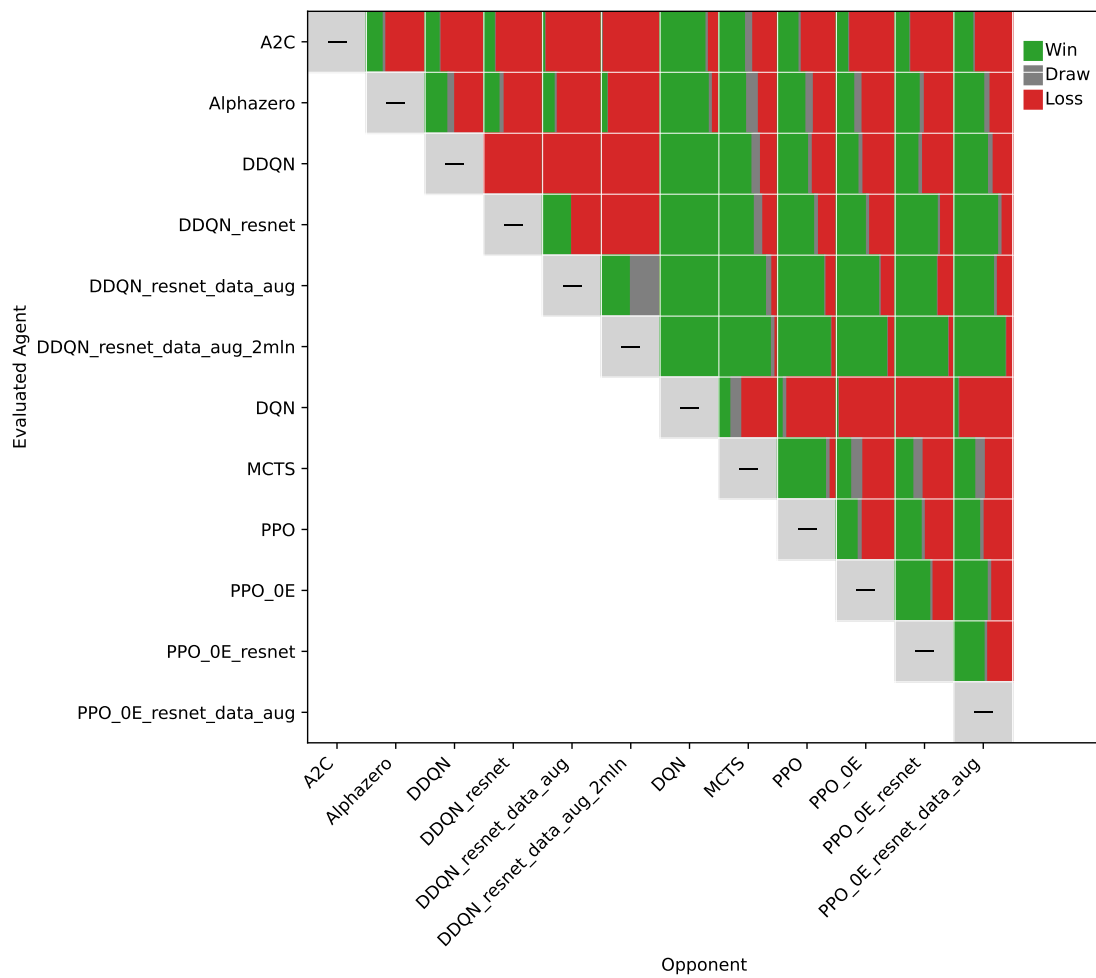


Figure 7.19: Empirical outcome frequencies for each agent pairing, where rows represent the evaluated agent and columns represent the opponent. Each cell visualizes the proportion of results—green for wins, gray for draws, and red for losses—computed over 2000 games per matchup. The newly added agent is DDQN_resnet_data_aug_2mIn.

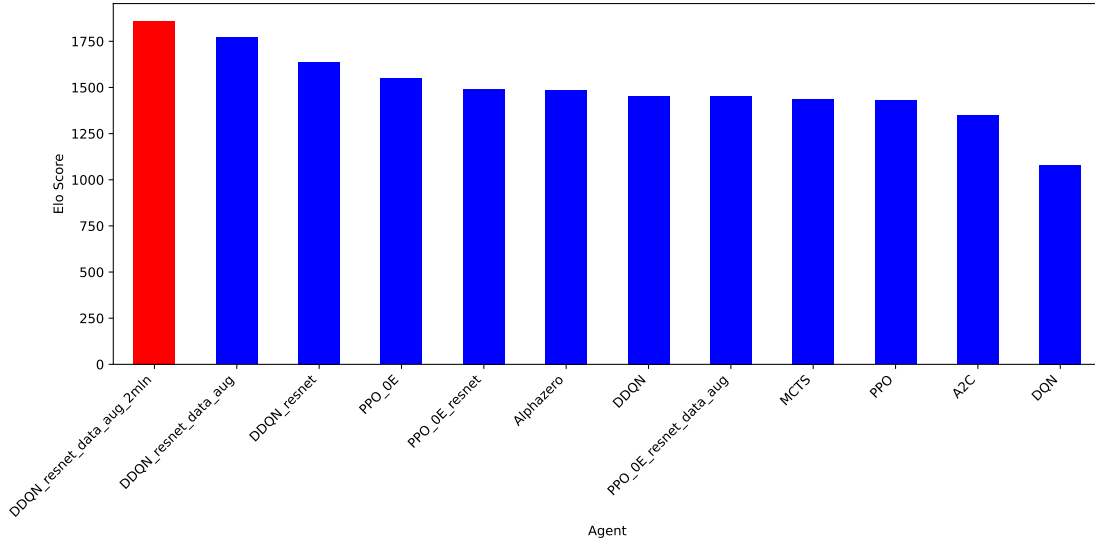


Figure 7.20: Final Elo ratings by agent, sorted from highest to lowest. In red is the top-performing model: DDQN_resnet_data_aug_2m1n.

This final training iteration confirms that DDQN_resnet_data_aug_2m1n is the most capable agent developed in this study with a final Elo score of 1861.4. By combining architectural enhancements, data augmentation, and extended training, the model consistently outperformed the other contenders in both head-to-head matchups and overall Elo ranking. These results demonstrate that thoughtful integration of model design and training methodology can yield significant improvements in strategic performance, even within the constraints of discrete-action environments like Ultimate Tic Tac Toe.

7.3 BEST RESPONSE

The objective of this experiment was to develop a robust agent when playing Ultimate Tic Tac Toe. As discussed in detail in Section 5, an agent is said to have reached a Nash equilibrium if, when playing against its best response (in simple words, its most challenging opponent), it achieves a combined win and draw rate of at least 50%. To evaluate this aspect, and therefore its robustness, we trained a PPO agent with zero entropy, intended to approximate the best response, against our strongest model, DDQN_resnet_data_aug_2m1n. We then let them play 1,000 games every 1,000 iterations to assess the win, draw, and loss rates of our best agent.

It is important to note that the best PPO-based response is only an approximation of the true best response, as it is learned through a neural network. The true best response would

ideally be a min-max strategy, but due to the immense size of the game tree in Ultimate Tic Tac Toe, computing a true min-max solution is infeasible given our hardware constraints.

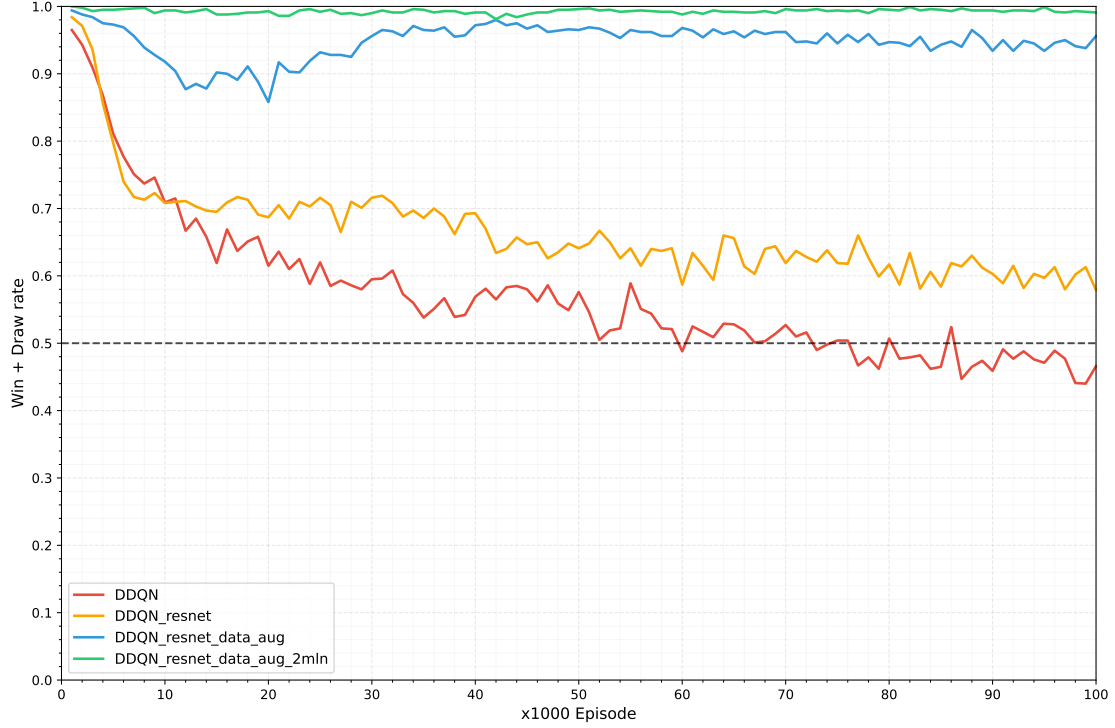


Figure 7.21: This chart represents the win and draw rate summed of the agents showed in the legend, different colors for different agents, against the best response under training every 1,000 iterations.

From Figure 7.21 we can observe that, throughout training, our DDQN_resnet_data_aug_2mln agent (blue curve) maintains a combined win+draw rate well above the 50% threshold required to approximate a Nash equilibrium. Even as the PPO best-response (orange curve) improves, it fails to push the DDQN below this 50% line, confirming the robustness of our final model.

To further validate against a near-minimax opponent, we evaluated DDQN_resnet_data_aug_2mln against a Monte Carlo Tree Search (MCTS) agent using 1,000 simulations (four times the number of simulations used during the free-for-all tournament) per move over 200 games (100 as first player, 100 as second). The results are summarized in Table 7.2.

	Win	Draw	Loss
DDQN_resnet_data_aug_2m1n vs MCTS-1000	0.775	0.135	0.090

Table 7.2: Performance of the final DDQN model against an MCTS agent with 1,000 simulations.

The combined win+draw rate of $0.775 + 0.135 = 0.910$ is much higher than the 0.50 threshold, even when faced with an agent that closely approximates a min-max strategy. Taken together, these experiments demonstrate that DDQN_resnet_data_aug_2m1n not only withstands a learned policy-gradient best response, but also dominates a strong search-based opponent, strongly suggesting that it has reached an approximate Nash equilibrium for Ultimate Tic Tac Toe within our computational constraints.

7.4 DISCUSSION OF RESULTS

7.4.1 ANALYSIS

This section interprets the empirical results presented in the previous sections, with a focus on efficacy, sample efficiency, architectural attributes, and robustness. We discuss why residual networks benefited DDQN but not PPO, the observed stability improvements, and how data augmentation contributed to faster convergence. We also reflect on the agent’s behavior in corner cases, based on both quantitative metrics and our interactive evaluation through the web demo (available at uttt-unipd.vercel.app).

As shown in Table 7.20, the Elo progression of our agents reflects the cumulative impact of architectural and procedural enhancements. The baseline DQN stabilized around 1079 Elo, offering limited resistance to stronger opponents. Incrementally, by adopting DDQN, introducing data augmentation based on board rotations, and deepening the network via residual connections, we achieved a final Elo near 1861 with the model DDQN_resnet_data_aug_2m1n. This corresponds to a 72% relative improvement and translates into winning approximately 85% of matches while maintaining a loss rate below 10% across a diverse pool of opponents. These results not only highlight the importance of architectural design but also demonstrate that modern value-based methods, when carefully engineered, can attain high performance even in non-trivial strategic environments.

One of the most influential changes was the integration of residual networks (ResNet). Residual connections enable the training of deeper models by facilitating gradient flow and miti-

gating the vanishing gradient problem, which is especially critical when the input space, such as a 9x9 game board, requires hierarchical abstraction. The residual architecture allowed the DDQN agent to learn more abstract, spatially invariant features, effectively capturing long-range dependencies between micro-board dynamics and macro-board outcomes. Interestingly, when we introduced the same ResNet architecture within our PPO implementation, we did not observe comparable gains. This discrepancy may be attributed to the inherent differences between value-based and actor-critic algorithms. DDQN, being off-policy, benefits from more stable training targets and can exploit deeper networks without destabilizing policy updates. PPO, on the other hand, is more sensitive to network depth due to its on-policy nature and the high variance in advantage estimation. The introduction of a deeper network in PPO probably worsen these instabilities, leading to suboptimal learning dynamics or premature policy collapse.

In addition to quantitative evaluations, we developed a web-based interface that allows users to play against our trained agents (publicly available at uttt-unipd.vercel.app). This interactive demo serves a dual purpose: on one hand, it acts as a tangible validation of the agent’s strategic competence, allowing human players to qualitatively assess the AI’s decisions and mistakes. On the other hand, it provides an educational tool for those interested in reinforcement learning, enabling them to visualize how learned policies behave in a high-variance, partially strategic environment. Importantly, we also used the platform ourselves throughout the development process to gain qualitative insights into the strengths and weaknesses of the models, particularly in corner cases where quantitative metrics alone offered limited visibility. This hands-on analysis proved valuable for identifying failure modes, spotting misaligned incentives and guiding us towards further training adjustments.

7.4.2 FUTURE WORKS

Despite the encouraging results, several limitations emerged during the project. Our original ambition included training MuZero agents, combining model-based planning with learned dynamics. However, even with aggressive pruning of hyperparameters and architectural complexity, the compute requirements of MuZero exceeded what was feasible under our hardware budget. Training a single MuZero configuration to convergence would have demanded several hundred GPU hours, far beyond what we allocated for this thesis. However, this experience illuminated valuable directions for future research. Lightweight variants such as EfficientZero or Sampled MuZero could offer similar benefits with reduced cost and merit systematic in-

vestigation. Alternatively, planning-lite strategies such as integrating oversimplified MCTS at inference time, using the learned value function to guide rollouts, could yield hybrid agents that combine the best of value estimation and tree search.

Another promising direction involves reward design. In this work, we deliberately avoided reward shaping to preserve the purity of the learning signal and prevent specification bias. However, introducing intermediate rewards such as micro-board control, line completions, or tempo advantages, could accelerate early stage learning. Careful experimentation with these heuristics, possibly in a curriculum learning setting, may yield agents that reach high performance with fewer environment interactions.

Finally, meta-learning approaches such as Population-Based Training (PBT) could be employed to adapt hyperparameters and architectural choices on the fly. In dynamic environments like UT TT, PBT may help escape local optima by co-evolving agent policies and training schedules. Similarly, explainability techniques like saliency mapping or attention heatmaps could be integrated to study which board regions or features drive action selection, improving both transparency and pedagogical value.



Conclusions

This final chapter consolidates the general contributions of the work, articulates its theoretical and practical implications, and reflects on its relevance in the reinforcement learning landscape. While much of the technical evaluation was covered in Chapter 7, here we contextualize those results and extract general insights.

The decision to focus on Ultimate Tic-Tac-Toe was grounded in its unique position as a mid-complexity environment, sufficiently rich to challenge state-of-the-art algorithms, yet compact enough to support systematic experimentation. The hierarchical structure of the game and the variable action space offer a controlled workspace to assess learning stability, robustness to adversaries, and generalization between variants. Unlike real-world domains, games allow for fast simulations and reproducible benchmarks, making them ideal for iterative development and technical studies.

Importantly, this work did not end with performance metrics alone. By building and using an interactive web platform to play against trained agents, we gained valuable qualitative insight into agent strengths and weaknesses. This direct involvement complemented quantitative evaluations and helped identify strategic blind spots, such as difficulties in corner domination or final move timing. The platform also demonstrates that reinforcement learning can produce interpretable and reproducible systems with which users can interact directly, bridging the gap between academic research and practical deployment.

The potential relevance of this research is underscored by the real-world transferability of game-derived algorithms. Reinforcement learning agents are often trained in games because

these environments provide controlled, repeatable, and richly structured settings where learning and decision-making strategies can be rigorously tested. A striking example of this transferability is the Deep Q-Network (DQN), originally developed to master Atari games, which now informs sepsis treatment policies in Intensive Care Unit settings through the AI Clinician framework [26, 27, 28]. Developed at Imperial College London, the AI Clinician was trained on over 40,000 ICU patient records to learn optimal dosing strategies for intravenous fluids and vasopressors, using off-policy reinforcement learning. It demonstrated the ability to recommend treatment strategies that align with expert-level decision-making and could potentially improve long-term patient outcomes. This evolution from entertainment to life-critical decision-making illustrates that mastering games is not an end in itself, but a stepping stone toward building robust, general-purpose algorithms with real-world impact.

Furthermore, several limitations emerged along the way. The compute budget constrained our ability to experiment with more ambitious agents such as MuZero, which remains a promising candidate for future exploration—perhaps in its more lightweight variants like EfficientZero. Likewise, we intentionally avoided dense reward shaping to keep training unbiased, but future work may revisit this with caution, possibly in curriculum learning or hybrid reward setups. Population-based training and explainable RL also offer promising directions for making agents both more performant and more interpretable.

In summary, this thesis offers a complete pipeline for building, evaluating, and deploying deep RL agents in a moderately complex environment, producing both strong empirical results and transferable methodological insights. It confirms that combining principled design with modern deep learning tools enables robust and generalizable policies, even in the absence of search. The lessons learned here extend over Ultimate Tic-Tac-Toe, contributing to a deeper understanding of what makes reinforcement learning work in strategically rich, resource-constrained settings.

References

- [1] B. F. Skinner, *Science and Human Behavior*. Macmillan, 1953.
- [2] M. Minsky, “Theory of neural-analog reinforcement systems and its application to the brain model problem,” Ph.D. dissertation, Princeton University, 1954.
- [3] R. Bellman, *Dynamic Programming*. Princeton University Press, 1957.
- [4] R. A. Howard, *Dynamic Programming and Markov Processes*. MIT Press, 1960.
- [5] P. J. Werbos, “Beyond regression: New tools for prediction and analysis in the behavioral sciences,” Ph.D. dissertation, Harvard University, 1974.
- [6] A. G. Barto, R. S. Sutton, and C. W. Anderson, “Neuronlike adaptive elements that can solve difficult learning control problems,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 13, no. 5, pp. 834–846, 1981.
- [7] R. S. Sutton, “Learning to predict by the methods of temporal differences,” *Machine Learning*, vol. 3, no. 1, pp. 9–44, 1988.
- [8] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. MIT Press, 1998.
- [9] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, “Playing atari with deep reinforcement learning,” *arXiv preprint arXiv:1312.5602*, 2013.
- [10] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, and D. Hassabis, “Mastering the game of go with deep neural networks and tree search,” *Nature*, vol. 529, no. 7587, pp. 484–489, 2016.

- [11] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, Y. Chen, T. Lillicrap, F. Hui, L. Sifre, G. van den Driessche, T. Graepel, and D. Hassabis, “Mastering chess and shogi by self-play with a general reinforcement learning algorithm,” *arXiv preprint arXiv:1712.01815*, 2017.
- [12] J. Schrittwieser, I. Antonoglou, T. Hubert, K. Simonyan, L. Sifre, S. Schmitt, A. Guez, E. Lockhart, D. Hassabis, T. Graepel, T. Lillicrap, and D. Silver, “Mastering atari, go, chess and shogi by planning with a learned model,” *arXiv preprint arXiv:1911.08265*, 2019. [Online]. Available: <https://arxiv.org/abs/1911.08265>
- [13] OpenAI, “Chatgpt: Optimizing language models for dialogue,” *OpenAI Blog*, 2022. [Online]. Available: <https://openai.com/blog/chatgpt/>
- [14] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray *et al.*, “Training language models to follow instructions with human feedback,” *arXiv preprint arXiv:2203.02155*, 2022.
- [15] Z. M. K. Zuhri, “A combinatorial analysis of tic-tac-toe and the theoretical advantage of playing first,” [https://informatika.stei.itb.ac.id/~rinaldi.munir/Matdis/2021-2022/Makalah2021/Makalah-Matdis-2021%20\(148\).pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Matdis/2021-2022/Makalah2021/Makalah-Matdis-2021%20(148).pdf), Bandung, Indonesia, 2021, program Studi Teknik Informatika, Sekolah Teknik Elektro dan Informatika.
- [16] X. Olivia, “At most 43 moves, at least 29: An optimal winning strategy for ultimate tic-tac-toe,” *arXiv preprint arXiv:2006.02353*, 2020. [Online]. Available: <https://arxiv.org/abs/2006.02353>
- [17] A. Nowaczyński, “Ultimate tic-tac-toe ai project,” 2021. [Online]. Available: <https://www.utt.ai/>
- [18] B. Addison, M. Peeler, and C. Alvin, “Ultimate tic-tac-toe bot techniques,” in *The International FLAIRS Conference Proceedings*, vol. 35, 2022.
- [19] H. Van Hasselt, A. Guez, and D. Silver, “Deep reinforcement learning with double q-learning,” *arXiv preprint arXiv:1509.06461*, 2015. [Online]. Available: <https://arxiv.org/abs/1509.06461>
- [20] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017. [Online]. Available: <https://arxiv.org/abs/1707.06347>

- [21] R. Coulom, “Efficient selectivity and backup operators in monte-carlo tree search,” in *Computers and Games: CG 2006*, ser. Lecture Notes in Computer Science, H. J. van den Herik, P. Ciancarini, and H. H. L. M. Donkers, Eds. Berlin, Heidelberg: Springer, 2006, vol. 4630, pp. 72–83. [Online]. Available: https://doi.org/10.1007/978-3-540-75538-8_7
- [22] J. v. Neumann, “Zur theorie der gesellschaftsspiele,” *Mathematische Annalen*, vol. 100, pp. 295–320, 1928. [Online]. Available: <https://doi.org/10.1007/BF01448847>
- [23] J. von Neumann and O. Morgenstern, *Theory of Games and Economic Behavior*. Princeton: Princeton University Press, 1944.
- [24] “Openai gym,” <https://www.gymnasium.dev>, 2016, accessed: 2024-12-29.
- [25] E. D’Albertyon, “Master thesis: Reinforcement learning agents for ultimate tic tac toe,” 2024, gitHub repository with code and resources used for the master thesis. [Online]. Available: https://github.com/enricopro/master_thesis
- [26] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu *et al.*, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [27] M. Komorowski, L. A. Celi, O. Badawi, A. C. Gordon, and A. A. Faisal, “The artificial intelligence clinician learns optimal treatment strategies for sepsis in intensive care,” *Nature Medicine*, vol. 24, no. 11, pp. 1716–1720, 2018.
- [28] A. Raghu, M. Komorowski, I. Ahmed, L. A. Celi, P. Szolovits, and M. Ghassemi, “Continuous state-space models for optimal sepsis treatment—a deep reinforcement learning approach,” in *Machine Learning for Healthcare Conference*. PMLR, 2017, pp. 147–163.

